

Planeación de acciones para robots de servicio
mediante modelos de lenguaje y sistemas basados en
reglas

Planeación de acciones para robots de servicio a
partir de instrucciones en lenguaje natural mediante
Qwen, CLIPS y modelos GPT

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción	1
1.1. Contexto y Motivación	2
1.2. Planteamiento del problema	3
1.3. Hipótesis	4
1.4. Objetivos	5
1.5. Estructura del documento	6
2. Antecedentes y Estado del Arte	8
2.1. Sistemas Basados en Reglas	8
2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones	9
2.2.1. Características Fundamentales de CLIPS	9
2.3. Modelos de lenguaje para interpretación de instrucciones robóticas	11
2.3.1. Principales Paradigmas de Aprendizaje Automático en Robótica	11
2.3.2. Familia de modelos Qwen	12
2.3.3. Modelos GPT	13
2.3.4. Interpretación semántica y generación de planes robóticos	13
2.3.5. Limitaciones de los modelos de lenguaje en robótica	14
2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA	14
2.3.7. Arquitecturas neuro-simbólicas	15
2.3.8. Conexión con la Propuesta de Tesis	17
3. Marco Teórico y Conceptual	19
3.1. Arquitectura de un Sistema de Planeación de Acciones Jerárquico	23
3.1.1. Fundamentos de la Planificación Jerárquica: HTN	24
3.1.2. El Planificador HATP (Hierarchical Agent-Based Task Planner)	25
3.1.3. Taxonomía de Arquitecturas Jerárquicas en Robótica	25

3.1.4.	Relevancia para el Sistema Propuesto	26
3.2.	Fundamentos de los Sistemas de Producción (Rule-Based Systems) . .	27
3.2.1.	Estructura y Componentes de un Sistema de Producción	27
3.2.2.	El Ciclo Reconocer-Actuar	28
3.2.3.	El Algoritmo RETE	28
3.2.4.	Implementación de RETE en CLIPS	30
3.2.5.	Ventajas y Limitaciones de los Sistemas de Producción	31
3.3.	Introducción a los Modelos de Lenguaje Grande (LLMs) y ChatGPT/Qwen	32
3.3.1.	Transformers y Mecanismo de Atención	33
3.3.2.	ChatGPT: Modelo Propietario para Interacción Conversacional	36
3.3.3.	Qwen2.5-0.5B: Modelo Open-Source para Cómputo	37
3.3.4.	Comparativa y Selección para Robótica de Servicio	39
3.4.	Integración de Traductores de Lenguaje Natural con Sistemas Basados en Reglas	40
3.4.1.	Utilidad y Función de la Integración	41
3.4.2.	Funcionamiento de la Integración CLIPS-Lenguaje Natural . . .	42
3.4.3.	Comparativa y Roles en la Arquitectura Híbrida	44
3.5.	ChatGPT como Agente de Planificación Autónomo	45
3.5.1.	Capacidades de Planificación de LLMs vs. Sistemas Basados en Reglas	46
3.5.2.	Arquitectura de Planificación Dual: CLIPS y ChatGPT como Mo- tores Complementarios	48
3.5.3.	Mecanismos de Selección y Validación de Planes	51
3.5.4.	Ventajas y Riesgos de la Planificación con LLMs en Robótica . .	53
4.	Metodología: Diseño del Sistema Híbrido CLIPS-ChatGPT/Qwen	56
4.1.	Diseño del Sistema Basado en Reglas con CLIPS	56
4.1.1.	Definición del Conjunto de Reglas Jerárquicas	56
4.1.2.	Priorización de Acciones Críticas y Gestión de Emergencias . . .	56
4.1.3.	Representación del Conocimiento y Hechos en la Base de Datos de CLIPS	56
4.2.	Integración de ChatGPT/Qwen como Sistema Complementario	57
4.2.1.	Funciones Asignadas al Módulo de Lenguaje Natural	57
4.2.2.	Protocolo de Comunicación entre CLIPS y los Modelos de Lenguaje	57

4.2.3. Mecanismos de Seguridad y Validación para las Respuestas del LLM	57
4.3. Implementación de Planificación Dual	57
4.3.1. Diseño del Módulo de Planificación con ChatGPT	57
4.3.2. Protocolos de Prompting para Generación de Planes Robóticos .	57
4.3.3. Mecanismo de Selección entre Planes CLIPS vs. ChatGPT . . .	57
4.3.4. Validación y Simulación de Planes Antes de Ejecución	57
5. Implementación	59
5.1. Entornos de Desarrollo: Simulación y Plataforma Física	59
5.2. Implementación del Motor de Reglas en CLIPS	59
5.3. Desarrollo del Módulo de Integración	59
5.4. Configuración de la Interfaz con API de OpenAI para ChatGPT o Modelo Qwen Local	59
5.5. Casos de Prueba para Validar la Interacción entre los Módulos	59
6. Escenarios de Validación y Experimentación	60
6.1. Diseño de Experimentos en Entornos Controlados	60
6.1.1. Escenario 1: Ejecución de Tareas Predefinidas (solo CLIPS) . .	60
6.1.2. Escenario 2: Gestión de Órdenes Imprecisas o Novedosas (CLIPS + ChatGPT/Qwen)	60
6.1.3. Escenario 3: Respuesta a Eventos Inesperados o Fallos	60
6.1.4. Escenario 4: Planificación Dual para Tareas Complejas (CLIPS vs. ChatGPT)	60
6.2. Métricas de Evaluación	61
6.2.1. Métricas Cuantitativas: Tiempo de Ejecución, Tasa de Éxito, Uso de Recursos Computacionales	61
6.2.2. Métricas Cualitativas: Robustez, Interpretabilidad y Fluidez en la Interacción Humano-Robot	61
7. Análisis de Resultados	62
7.1. Comparativa del Rendimiento del Sistema Solo-CLIPS vs. el Sistema Híbrido	62
7.2. Evaluación de la Efectividad de ChatGPT/Qwen en las Diferentes Funciones Asignadas	62

7.3. Discusión de Limitaciones y Errores	62
7.4. Análisis de la Escalabilidad y el Coste Computacional de la Integración	62
8. Discusión	63
8.1. Interpretación de los Resultados en el Contexto de los Objetivos	63
8.2. Ventajas y Desventajas del Enfoque Híbrido Propuesto	63
8.3. Implicaciones para la Seguridad y Certificación en Entornos Regulados	63
8.4. Límites Éticos y Prácticos del Uso de ChatGPT/Qwen en Robótica . .	63
9. Contribuciones y Relevancia	64
9.1. Contribución Técnica: Marco Híbrido Escalable y Documentado	64
9.2. Relevancia Práctica: Aplicaciones en Salud, Industria y Logística	64
9.3. Relevancia Académica: Benchmark para Sistemas Rule-Based vs. Híbridos	64
10. Conclusiones y Trabajo Futuro	65
10.1. Conclusiones Principales	65
10.2. Propuestas de Trabajo Futuro	65

Índice de figuras

3.1. Arquitectura ViRoot.	20
3.2. Arquitectura Funcional ViRoot.	21
3.3. Arquitectura de planificación dual CLIPS-ChatGPT.	49

Capítulo 1

Introducción

Los robots de servicio representan un campo de proyección para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1), donde deben interpretar instrucciones expresadas por personas y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción.(2).

La transformación de una instrucción en lenguaje natural a un plan robótico presenta dos problemas relacionados. El primero es semántico: identificar correctamente los objetivos, entidades, destinos y relaciones expresados por el usuario. El segundo es simbólico: convertir dicha interpretación en una secuencia de acciones pertenecientes al repertorio real del robot y con parámetros compatibles con el ejecutor.

Los sistemas basados en reglas, han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). En particular, CLIPS utiliza hechos y reglas de producción para determinar qué transformaciones deben aplicarse a partir de un estado simbólico. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística puede producir formatos inválidos, entidades inexistentes o acciones que no pertenecen a las capacidades del robot. Por lo tanto la falta de garantías formales de seguridad limitan

su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis estudia dos aproximaciones para resolver el problema. La primera es una arquitectura neuro-simbólica en la que un modelo Qwen2.5-7B, ajustado mediante QLoRA, transforma una orden en objetivos canónicos representados en JSON. Posteriormente, un módulo determinista normaliza y convierte esos objetivos en hechos, que son procesados por reglas de producción en CLIPS para generar el plan compatible con el ejecutor del robot. La segunda aproximación emplea un modelo GPT como planificador generativo directo. En este caso, el modelo recibe la orden, el contexto y el catálogo de acciones, produciendo una respuesta estructurada mediante JSON Schema que es revisada por un validador determinista antes de considerarse ejecutable.

Las dos aproximaciones no se combinan mediante un selector de planes durante la ejecución. Se implementan como arquitecturas independientes y se comparan experimentalmente bajo el mismo conjunto de órdenes, contexto y criterios de evaluación. Esta comparación permite analizar el compromiso entre interpretación semántica, consistencia, trazabilidad, flexibilidad, latencia, recursos computacionales y coste de uso.

1.1. Contexto y Motivación

El área de la robotica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1).

El robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra módulos de percepción, navegación, manipulación e interacción mediante ROS 2. Para ejecutar una orden, el robot necesita una representación intermedia que conecte el lenguaje del usuario con las acciones ofrecidas por dichos módulos.

Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas (*Rule-Based Systems, RBS*), que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, una solución exclusivamente

simbólica puede generar planes consistentes cuando recibe objetivos correctamente estructurados, pero no resuelve directamente la interpretación del lenguaje natural. En contraste, un modelo de lenguaje puede interpretar distintas formulaciones de una orden, aunque su salida no es necesariamente compatible con las restricciones del robot. La motivación de este trabajo es evaluar si la separación entre interpretación neuronal y expansión simbólica permite obtener planes ejecutables y trazables, y contrastar sus resultados con los de un planificador basado exclusivamente en un modelo GPT.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se busca identificar que arquitectura presenta ventajas dependiendo del comando, así como que errores introduce cada etapa y cuáles son los costes asociados con su despliegue local o mediante una API externa.

1.2. Planteamiento del problema

Una orden puede contener variaciones lingüísticas, referencias espaciales, objetos, personas, destinos y varias subtarefas relacionadas. Para que el robot pueda ejecutarla, la orden debe convertirse en una secuencia válida de acciones primitivas, para esto el sistema basados en reglas CLIPS, fundamentado en la lógica simbólica y el ciclo *reconocer-actuar* (3), usa un conjunto de hechos y reglas bien definidos para producir una respuesta determinista y explicable, lo que lo hace idóneo para aplicaciones donde la seguridad es crítica, como entornos industriales o médicos.

La conversión por parte de CLIPS enfrenta los siguientes problemas:

1. **Rigidez interpretativa** No se pueden procesar comandos en lenguaje natural complejo o ambiguo.
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los resultados deben satisfacer al menos las siguientes condiciones (6):

- Las entidades finales deben pertenecer al catálogo o ser tratadas explícitamente como desconocidas.

- Los objetivos deben conservar el significado y el orden causal de la instrucción.
- Las acciones generadas deben pertenecer al repertorio del ejecutor.
- Cada acción debe contener los parámetros requeridos.
- La secuencia debe respetar precondiciones simbólicas básicas, como la ubicación del robot o la posesión de un objeto.

Los modelos de lenguaje pueden ampliar la cobertura lingüística, aunque también pueden generar resultados inexistentes o inconsistentes. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Qué diferencias de exactitud, consistencia, robustez y coste existen entre una arquitectura neuro-simbólica Qwen-CLIPS y un planificador generativo basado en GPT al convertir órdenes GPSR en planes para un robot de servicio?

1.3. Hipótesis

Para la interpretación y planeación de órdenes, una arquitectura neuro-simbólica compuesta por Qwen y CLIPS producirá una mayor proporción de planes estructuralmente válidos, trazables y consistentes que un planificador generativo basado directamente en GPT. Por otra parte, se espera que el planificador GPT directo presente mayor flexibilidad ante formulaciones lingüísticas novedosas, pero con mayor variabilidad, latencia y coste de inferencia mediante API.

Se espera que los sistemas:

- Mejore la tasa de éxito en la ejecución de tareas en entornos domésticos.
- Reduzca el tiempo de respuesta ante comandos complejos.
- Mantenga un comportamiento seguro y predecible en situaciones críticas.
- Permita una interacción más natural e intuitiva con usuarios no expertos.

La hipótesis se evaluará de forma separada para la interpretación de objetivos y para la generación del plan completo. De esta manera será posible determinar si un error proviene del modelo de lenguaje, de la conversión simbólica o de la cobertura de las reglas CLIPS.

1.4. Objetivos

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con un planificador generativo basado en GPT con salida estructurada.

■ Objetivos específicos

- Definir una representación canónica de objetivos y su correspondencia con las acciones disponibles en el ejecutor.
- Ajustar Qwen2.5-7B mediante QLoRA para convertir órdenes en objetivos canónicos representados en JSON.
- Implementar la normalización, validación y conversión de los objetivos generados por Qwen en hechos.
- Implementar reglas de producción en CLIPS que expandan objetivos de alto nivel en secuencias de acciones compatibles con el ejecutor.
- Diseñar un planificador GPT directo mediante un prompt, un JSON Schema y un validador determinista.
- Construir un conjunto experimental que incluya órdenes conocidas, paráfrasis, composiciones de objetivos, entradas ambiguas y casos fuera del dominio.
- Comparar las arquitecturas en términos de exactitud semántica, validez estructural, robustez, latencia, consumo de recursos, variabilidad y coste.
- Validar en simulación, robot físico y durante la RoboCup 2026 el resultado los planes generados.

■ Contribuciones esperadas

- Una representación intermedia de objetivos que conecta la interpretación lingüística con la planeación simbólica.
- Un modelo Qwen2.5-7B ajustado mediante QLoRA para traducir órdenes a objetivos canónicos.
- Una integración ROS 2–CLIPS que convierte dichos objetivos en planes compatibles con el ejecutor.
- Un planificador GPT directo restringido mediante prompt, JSON Schema y validación determinista.
- Un protocolo experimental reproducible para comparar planeación neuro-simbólica y planeación generativa.
- Un análisis de errores por etapa que diferencia fallos semánticos, estructurales, simbólicos y de ejecución.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y validación del sistema híbrido de planificación. La estructura es la siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.
3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología.** Detalla el diseño del sistema híbrido, incluyendo la definición de reglas en CLIPS, la integración con ChatGPT/Qwen, y los mecanismos de planificación dual y validación.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Análisis de resultados.** Compara el rendimiento del sistema híbrido frente al sistema basado solo en reglas, evalúa la efectividad de los LLMs y discute limitaciones y costos computacionales.
8. **Capítulo 8: Discusión.** Interpreta los resultados en relación con los objetivos, analiza ventajas y desventajas del enfoque híbrido, y explora implicaciones éticas y de seguridad.
9. **Capítulo 9: Contribuciones y relevancia.** Destaca la aportación técnica del marco híbrido, su aplicabilidad práctica en distintos sectores y su relevancia académica como benchmark.
10. **Capítulo 10: Conclusiones y trabajo futuro.** Resume los hallazgos principales y propone líneas de investigación futura, como el fine-tuning de LLMs y la

mejora de mecanismos de seguridad.

Capítulo 2

Antecedentes y Estado del Arte

Este capítulo presenta los trabajos y tecnologías relacionados con la transformación de instrucciones de lenguaje natural a planes para robots de servicio. La revisión se concentra en cuatro componentes relevantes para la tesis: sistemas de producción, CLIPS, modelos de lenguaje para interpretación y planeación, y arquitecturas neuro-simbólicas. Los métodos de percepción, navegación o control se mencionan únicamente cuando contribuyen directamente a la interfaz entre la orden y el plan de tareas.

2.1. Sistemas Basados en Reglas

Los sistemas basados en reglas (*Rule-Based Systems, RBS*), constituyen uno de los paradigmas más antiguos y consolidados en inteligencia artificial (3). Su origen se remonta a los sistemas expertos de la década de 1970, donde se utilizaban para representar el conocimiento de especialistas humanos en dominios bien delimitados, como el diagnóstico médico o la configuración de sistemas complejos (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales estructurados, donde la previsibilidad y seguridad eran prioritarias.

Históricamente, los RBS han sido fundamentales en arquitecturas de control robótico jerárquico, como la propuesta por Brooks (4), donde capas de comportamientos reactivos podían ser coordinadas mediante reglas de supresión. Más adelante, motores de inferencia como CLIPS (*C Language Integrated Production System*), desarrollado por la NASA en 1985, se convirtieron en herramientas estándar para la implementación de sistemas de planificación y toma de decisiones en robots autónomos (7). CLIPS permitió la representación simbólica del conocimiento a través de hechos, reglas, en

conjunto con un ciclo de inferencia (*reconocer-actuar*), que ofreció un marco eficiente para el razonamiento en tiempo real.

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** En cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** En tareas simples como la entrega de objetos, navegación en interiores y asistencia a personas con movilidad reducida.
3. **Robótica espacial y de exploración:** Sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).

Su principal ventaja para esta tesis es que permiten establecer una correspondencia explícita entre un objetivo simbólico y las acciones emitidas para el ejecutor. Sin embargo, la cobertura del sistema está limitada por las reglas y los tipos de hechos definidos previamente.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en LISP y en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS es un entorno para el desarrollo de sistemas basados en conocimiento que ofrece hechos, plantillas, reglas de producción, funciones y un motor de inferencia que

permite razonar sobre una base de hechos utilizando reglas del tipo if-then (3).

En el contexto de esta tesis, las propiedades relevantes de CLIPS son:

1. **Encadenamiento hacia adelante (*forward chaining*):** CLIPS utiliza predominantemente encadenamiento hacia adelante, lo que significa que, partiendo de un conjunto de hechos iniciales, el motor de inferencia aplica reglas para derivar nuevos hechos de manera iterativa (7). Este enfoque es especialmente adecuado para aplicaciones en tiempo real donde los cambios en el entorno deben reflejarse inmediatamente en la base de conocimiento (1).
2. **Representación del conocimiento multifacética:** Además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas (*def-template*), así como mediante funciones definidas por el usuario en C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (9).
3. **Trazabilidad de las reglas activadas:** El comportamiento es reproducible cuando se mantienen constantes los hechos, las reglas y la estrategia de resolución de conflictos.
4. **Portabilidad y código abierto:** Escrito en C, CLIPS puede ejecutarse en prácticamente cualquier plataforma debido a su bajo costo, desde microcontroladores hasta sistemas de alto rendimiento (3). Su naturaleza open source ha permitido su adaptación en múltiples contextos robóticos, como el control de robots de exploración y sistemas de inspección (7).
5. **Integración con otros lenguajes:** CLIPS ofrece una API en C y una librería en python, lo que le permite ser empleado en aplicaciones más grandes, posibilitado su integración con frameworks robóticos como ROS y entornos de simulación (1).

Estas propiedades no implican por sí mismas seguridad garantizada ni verificación formal. Una base CLIPS puede contener reglas incompletas o contradictorias, y su comportamiento depende de la cobertura del conocimiento y de la resolución de conflictos. En consecuencia, esta tesis utiliza CLIPS como mecanismo explícito y trazable de expansión de objetivos, no como prueba formal de seguridad.

CLIPS tampoco interpreta directamente lenguaje natural ni aprende reglas a partir de los ejemplos empleados en esta investigación. Para recibir una orden requiere una etapa externa que produzca objetivos estructurados. Además, la integración con ROS 2 no es una capacidad intrínseca de CLIPS, sino una interfaz implementada específicamente para comunicar el motor de reglas con los nodos del robot.

2.3. Modelos de lenguaje para interpretación de instrucciones robóticas

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha transformado radicalmente la robótica contemporánea, habilitando capacidades que eran difíciles de lograr con enfoques puramente simbólicos o geométricos (10). A diferencia de los sistemas basados en reglas, que requieren la especificación explícita de todo el conocimiento del dominio, los enfoques de ML permiten a los robots aprender directamente de los datos, extrayendo patrones complejos y adaptándose a entornos variables sin necesidad de programación explícita (2).

2.3.1. Principales Paradigmas de Aprendizaje Automático en Robótica

Aprendizaje Supervisado y Visión por Computadora: Las redes neuronales convolucionales (CNNs) se han convertido en el estándar de facto para tareas de percepción robótica, incluyendo detección y reconocimiento de objetos, segmentación semántica y estimación de poses (11). Arquitecturas como YOLO (You Only Look Once) y sus variantes permiten a robots como Justina identificar objetos en tiempo real con alta precisión, incluso en entornos domésticos desordenados. Las CNNs operan mediante capas de convolución que extraen jerárquicamente características visuales, desde bordes simples hasta formas complejas, permitiendo una representación robusta del entorno (11).

Aprendizaje por Refuerzo para Control y Navegación: El aprendizaje por refuerzo profundo (Deep RL) ha emergido como una herramienta poderosa para desarrollar políticas de control en tareas secuenciales. Mediante la interacción con el entorno, los agentes aprenden a maximizar recompensas acumulativas, descubriendo estrategias óptimas para navegación, manipulación y evitación de obstáculos (12). En navegación robótica, se han desarrollado políticas híbridas que combinan representaciones aprendidas con modelos geométricos tradicionales para operar robustamente en entornos reales (12).

Redes Recurrentes y Modelado Secuencial: Las redes neuronales recurrentes (RNNs) y sus variantes (LSTM, GRU) son fundamentales para tareas que requieren memoria temporal, como el seguimiento de objetos en movimiento, la predicción de

trayectorias humanas o la interpretación de comandos verbales en contexto (10). Estas arquitecturas mantienen un estado interno que evoluciona con la secuencia de entradas, permitiendo modelar dependencias temporales de largo alcance.

Modelos de Lenguaje Grande (LLMs) en Robótica: La irrupción de modelos transformadores, como los descritos por (5), han dado lugar a los Large Language Models (LLMs), capaces de procesar y generar lenguaje natural con una fluidez sin precedentes. En robótica, estos modelos se utilizan para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas, e incluso generar planes completos a partir de descripciones de alto nivel (6). Su capacidad para captar matices semánticos y contextuales los hace especialmente valiosos para la interacción humano-robot en entornos domésticos.

2.3.2. Familia de modelos Qwen

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. La familia incluye modelos de distintos tamaños y variantes destinadas a tareas generales o al seguimiento de instrucciones (13). Estos modelos están basados en Transformers y pueden utilizarse para tareas como generación de texto, respuesta a preguntas, extracción de información y producción de contenido estructurado.

Qwen2.5 es una generación de esta familia que incluye modelos con diferentes números de parámetros. Para esta investigación se seleccionó ‘Qwen/Qwen2.5-7B’, un modelo de aproximadamente siete mil millones de parámetros.

En la arquitectura propuesta, Qwen2.5-7B funciona como intérprete semántico y no como generador del plan final. El modelo recibe una orden y produce una secuencia de objetivos canónicos representada en JSON. Esta secuencia identifica tipos de objetivo, entidades, ubicaciones y relaciones expresadas en la instrucción. Posteriormente, un módulo determinista normaliza y valida la respuesta antes de convertirla en hechos para CLIPS.

La selección de un modelo de siete mil millones de parámetros busca un compromiso entre capacidad lingüística y recursos computacionales. Además, su disponibilidad para ejecución en infraestructura controlada permite estudiar una alternativa local frente al consumo de un servicio externo. Estas ventajas no eliminan la necesidad de medir memoria, latencia y consumo computacional. Los detalles del despliegue, el conjunto de entrenamiento y los resultados pertenecen a los capítulos de implementación y evaluación.

2.3.3. Modelos GPT

Los modelos GPT constituyen otra familia de modelos generativos basados en Transformers. En esta tesis se accede al modelo seleccionado mediante la API de OpenAI, en lugar de desplegar sus parámetros en la infraestructura local. La API recibe las instrucciones y el contexto de la tarea y devuelve la respuesta generada por el modelo. El capítulo de implementación la versión utilizada, porque la denominación general “ChatGPT” no identifica de manera reproducible un modelo de API (14).

En la arquitectura experimental GPT directo, el modelo recibe la orden, el catálogo de entidades, el repertorio de acciones, las restricciones del dominio y ejemplos de respuesta. A diferencia de Qwen en la arquitectura neuro-simbólica, este modelo genera tanto la representación de los objetivos como la secuencia propuesta de acciones. Por ello, constituye un planificador generativo independiente y no un módulo interno de la cadena Qwen–CLIPS.

La respuesta se solicita mediante una salida estructurada definida con JSON Schema. La documentación de la API denomina *Structured Outputs* al mecanismo mediante el cual el modelo debe ajustar su respuesta al esquema proporcionado (14). Esta restricción permite comprobar la estructura de la respuesta, pero no demuestra que las entidades existan, que los parámetros sean apropiados ni que el plan sea físicamente seguro. Estas propiedades se comprueban mediante un validador determinista separado.

El uso de una API externa introduce variables que deben incluirse en la evaluación, como latencia de comunicación, disponibilidad del servicio, tokens de entrada y salida, coste monetario y posibles cambios entre versiones del modelo. En consecuencia, las pruebas deben fijar y documentar el modelo, los parámetros de generación y el prompt.

2.3.4. Interpretación semántica y generación de planes robóticos

Para esta tesis conviene distinguir dos funciones:

Interpretación semántica: Transformación de una orden en una representación intermedia de objetivos, entidades y relaciones.

Planeación generativa: Producción directa de una secuencia de acciones a partir de la orden, el contexto y las capacidades declaradas del robot.

La primera función se asigna a Qwen en la arquitectura neuro-simbólica. La segunda se estudia mediante un modelo GPT como condición experimental independiente. Esta

distinción permite comparar el efecto de conservar una etapa simbólica explícita frente a delegar al modelo tanto la interpretación como la composición del plan.

La interpretación semántica puede evaluarse comparando la secuencia de objetivos producida por el modelo con una anotación de referencia. La planeación requiere criterios adicionales: pertenencia de las acciones al repertorio del robot, validez de parámetros, conservación de dependencias, cumplimiento de precondiciones y compatibilidad con el ejecutor. Evaluar ambas etapas de forma separada permite localizar el origen de los errores.

2.3.5. Limitaciones de los modelos de lenguaje en robótica

El uso de un modelo de lenguaje introduce riesgos específicos: generación de entidades o acciones inexistentes, incumplimiento del formato, variabilidad entre ejecuciones, sensibilidad al *prompt* y costes de inferencia. Asimismo, una instrucción ambigua puede dar lugar a una interpretación plausible pero diferente de la intención del usuario.

Estas limitaciones son especialmente relevantes cuando la salida se utiliza para controlar un sistema físico. Por ello, una respuesta lingüísticamente coherente no debe considerarse automáticamente un plan ejecutable. En esta investigación se separan la generación, la validación estructural y la validación semántica; los módulos de ejecución del robot conservan la responsabilidad sobre restricciones físicas y condiciones observadas durante la tarea.

2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA

El ajuste completo de un modelo de lenguaje requiere actualizar todos sus parámetros y mantener estados adicionales durante el entrenamiento. Los métodos de adaptación eficiente en parámetros permiten reducir este coste al entrenar únicamente un subconjunto reducido de parámetros, manteniendo congelada gran parte del modelo preentrenado (15,16). LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas del modelo, manteniendo congelados los pesos originales (16), mientras que QLoRA combina estos adaptadores con la cuantización del modelo base para reducir significativamente el uso de memoria durante el ajuste fino (17).

En esta investigación se utiliza QLoRA para especializar Qwen2.5-7B en la conver-

sión de órdenes a objetivos canónicos. Los detalles del conjunto de datos, cuantización, hiperparámetros y plataforma pertenecen al capítulo de implementación; esta sección debe limitarse a explicar el fundamento y justificar la elección del método.

Salidas estructuradas y validación

Una estrategia para integrar modelos de lenguaje con software convencional consiste en restringir la salida a una estructura legible por máquina. JSON permite representar objetivos, parámetros y acciones, mientras que JSON Schema permite comprobar tipos, campos obligatorios, enumeraciones y restricciones estructurales.

No obstante, una respuesta conforme al esquema puede seguir siendo semánticamente incorrecta. Por ejemplo, puede incluir una ubicación que no pertenece al mapa o aplicar una acción válida al tipo de entidad equivocado. En consecuencia, la validación debe separar al menos tres niveles:

1. Sintaxis JSON.
2. Conformidad con el esquema.
3. Validez semántica respecto a catálogos, acciones y precondiciones.

Esta separación es aplicable tanto a los objetivos producidos por Qwen como al plan generado directamente por GPT.

“`latex`

2.3.7. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas buscan combinar las capacidades de aprendizaje y generalización de los modelos neuronales con mecanismos explícitos de representación de conocimiento y razonamiento simbólico. Mientras que los modelos neuronales son especialmente adecuados para procesar información no estructurada y aprender representaciones a partir de datos, los sistemas simbólicos permiten expresar conocimiento mediante estructuras discretas, reglas y relaciones que pueden ser utilizadas durante un proceso de inferencia (18, 19). El objetivo de este enfoque es aprovechar las fortalezas de ambos paradigmas en lugar de utilizar cada uno de manera aislada.

La integración entre ambos componentes puede realizarse de diferentes maneras. Un modelo neuronal puede, por ejemplo, transformar una entrada no estructurada en una representación simbólica que posteriormente es procesada por un sistema de razonamiento. En la dirección opuesta, el conocimiento simbólico puede utilizarse para

imponer restricciones, validar resultados o proporcionar información adicional durante el aprendizaje o la inferencia del modelo neuronal (19). Por tanto, una arquitectura neuro-simbólica no implica necesariamente que el aprendizaje neuronal y el razonamiento simbólico se realicen dentro de un único modelo; también pueden implementarse como componentes especializados conectados mediante representaciones intermedias.

Esta separación resulta particularmente útil cuando la entrada del sistema se encuentra expresada en lenguaje natural, pero la ejecución posterior requiere representaciones estructuradas y operaciones sujetas a reglas explícitas. En este tipo de arquitecturas, el modelo neuronal puede encargarse de interpretar la variabilidad lingüística y transformarla en una representación formal o canónica, mientras que un componente simbólico utiliza dicha representación para realizar razonamiento o planificación. Un esquema similar puede observarse en LLM+P, donde un modelo de lenguaje convierte la descripción de un problema de planificación expresado en lenguaje natural a PDDL y posteriormente delega la generación del plan a un planificador clásico (20).

La arquitectura principal propuesta en esta tesis sigue una estrategia de acoplamiento secuencial entre ambos paradigmas. En la primera etapa, un modelo de la familia Qwen2.5 (21), adaptado al dominio de tareas de servicio del robot, recibe una instrucción expresada en lenguaje natural y la transforma en una representación canónica de los objetivos solicitados. Esta etapa permite abstraer variaciones lingüísticas como diferentes estructuras gramaticales, sinónimos o formas alternativas de expresar una misma intención, produciendo una representación estructurada que puede ser consumida por el sistema de planificación.

En la segunda etapa, los objetivos obtenidos son enviados a CLIPS. A diferencia del modelo neuronal, CLIPS no interpreta directamente la instrucción original en lenguaje natural. Su función consiste en trabajar sobre los símbolos previamente obtenidos, incorporar los objetivos al estado de conocimiento y aplicar las reglas definidas para descomponerlos y expandirlos en las acciones necesarias para su ejecución. De esta manera, la interpretación lingüística y la expansión simbólica del plan permanecen como responsabilidades diferenciadas dentro de la arquitectura.

Esta división permite que el modelo neuronal se concentre en el problema de correspondencia entre lenguaje natural y representaciones canónicas, mientras que el conocimiento sobre la estructura de las tareas, sus relaciones y su descomposición se mantiene explícitamente en la base de reglas de CLIPS. En consecuencia, una modificación en las reglas de planificación no requiere necesariamente modificar el modelo de lenguaje y,

de forma análoga, las mejoras realizadas sobre el componente de interpretación pueden introducirse sin sustituir el mecanismo simbólico de planificación.

En esta tesis, por tanto, la integración neuro-simbólica se describe como una composición secuencial: Qwen transforma el lenguaje natural en objetivos canónicos y CLIPS expande dichos objetivos mediante reglas de producción. Los componentes no actúan como dos planificadores concurrentes ni se realiza una fusión automática de planes generados independientemente. El denominado planificador GPT directo se implementa como una arquitectura alternativa con fines de comparación experimental y no como un componente adicional de la cadena neuro-simbólica Qwen-CLIPS.

2.3.8. Conexión con la Propuesta de Tesis

Los antecedentes muestran que los sistemas simbólicos ofrecen una representación explícita del dominio, mientras que los modelos de lenguaje permiten procesar una mayor variedad de expresiones. Sin embargo, sigue siendo necesario medir de forma separada la calidad de la interpretación semántica y la ejecutabilidad del plan resultante bajo un mismo dominio, catálogo de entidades y repertorio de acciones.

Esta tesis aborda dicho problema mediante una comparación controlada entre una arquitectura secuencial Qwen-CLIPS y un planificador GPT directo. Adicionalmente, la condición de objetivos de referencia procesados por CLIPS permite estimar el rendimiento máximo de la etapa simbólica y distinguir los errores introducidos por la interpretación lingüística de aquellos causados por la cobertura de reglas.

La propuesta principal específicamente corresponde a una arquitectura neuro-simbólica de composición secuencial. Un modelo Qwen2.5-7B ajustado mediante QLoRA recibe una instrucción y produce una secuencia de objetivos canónicos representada en JSON. Un módulo determinista valida y normaliza la respuesta, la convierte en hechos y la envía al motor CLIPS. Finalmente, las reglas de producción expanden cada objetivo en acciones compatibles con el ejecutor del robot.

Como punto de comparación anteriormente mencionado se implementa un planificador basado en GPT que genera directamente una representación estructurada de los objetivos y el plan. Su salida se restringe mediante JSON Schema y se somete a validación determinista. Ambas arquitecturas se evalúan de forma independiente; por tanto, el trabajo no propone selección concurrente ni fusión automática de planes.

La contribución experimental consiste en comparar ambos enfoques bajo el mismo conjunto de órdenes, catálogos y acciones, separando los errores de interpretación de

los errores de planeación y cuantificando exactitud, ejecutabilidad, robustez, latencia, recursos y coste.

Capítulo 3

Marco Teórico y Conceptual

El fundamento teórico del sistema propuesto consiste en dos arquitectura de planeación. Se introduce el funcionamiento de los modelos de lenguaje grande y se detalla la integración de traductores de lenguaje natural con sistemas basados en reglas, estableciendo las bases tecnológicas para la implementación de los sistemas y su uso en la arquitectura ViRoot (*Virtual Real Robot*) descrita a continuación (ver Figura 3.1).

Arquitectura del Sistema

La arquitectura ViRBot proporciona el contexto funcional en el que se integra este trabajo. Incluye módulos de percepción, navegación, manipulación, reconocimiento y síntesis de voz, supervisión y ejecución 3.1). La tesis no modifica todos estos componentes; su alcance se concentra en la transformación de una orden textual GPSR en un plan aceptado por el ejecutor.

La arquitectura VIRBOT de Justina se organiza en cuatro capas principales:

■ Capa de Entradas

- Procesamiento de datos de sensores internos y externos.
- Aplicación de técnicas de reconocimiento de patrones.
- Generación del estado del entorno.

■ Capa de Planificación

- Validación mediante gestión del conocimiento.
- Reconocimiento de situaciones y activación de objetivos.
- Planificación de secuencias de operaciones físicas.

■ Capa de Gestión del Conocimiento

- Mapas del entorno creados con técnicas SLAM.

- Sistema de localización basado en filtro de Kalman.
- Sistema basado en reglas CLIPS para representación del conocimiento.
- **Capa de Ejecución**
 - Ejecución y verificación de planes de movimiento.
 - Procedimientos predefinidos representados como máquinas de estado.
 - Integración de procedimientos para generar planes complejos.

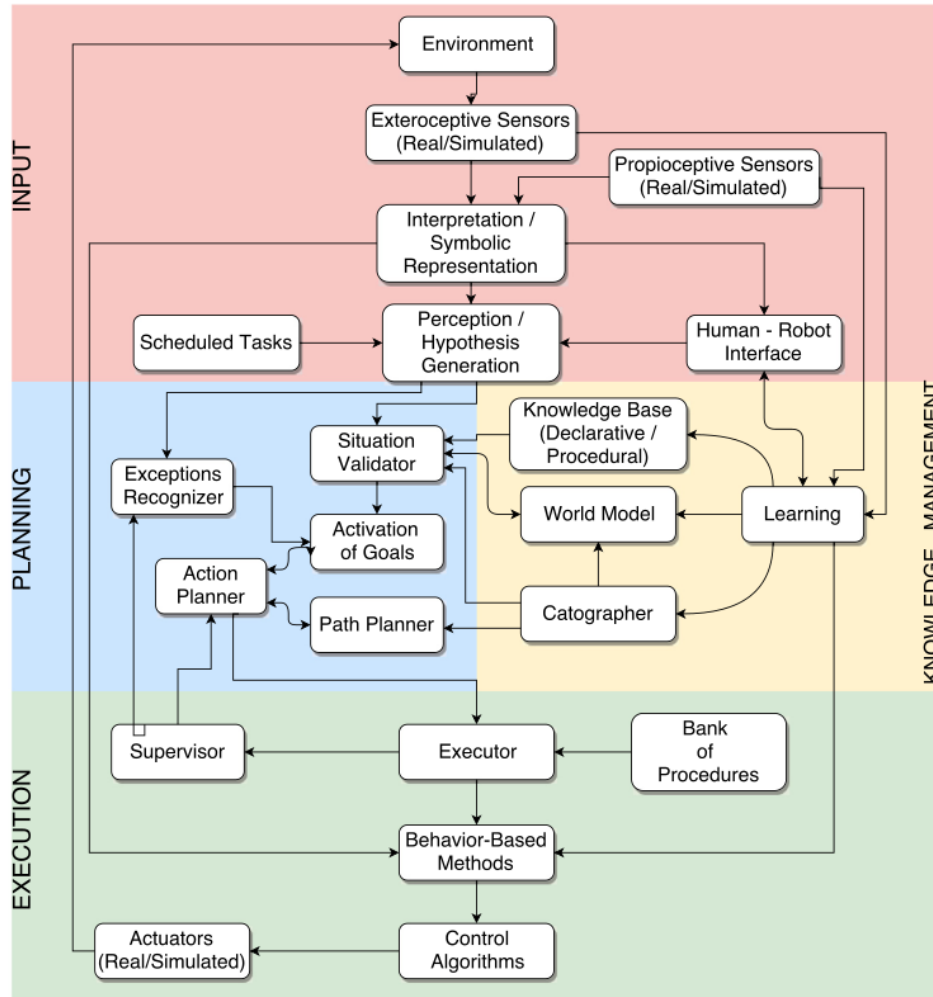


Figura 3.1: Arquitectura ViRoot.

Arquitectura Funcional de ViRbot

Esta arquitectura organiza los módulos funcionales del robot de manera interconectada, lo que permiten integrar la percepción, comprensión del lenguaje, planeación

simbólica y control físico del robot. La Figura 3.2 muestra la interacción entre los distintos subsistemas del robot Justina.

Debe distinguirse entre el estado interno utilizado durante la generación del plan y el estado confirmado durante la ejecución física. El planificador CLIPS mantiene un estado simbólico predictivo mínimo, por ejemplo la ubicación esperada del robot y el objeto que se espera que sostenga. Este estado se reinicia en cada ciclo de planeación y no constituye, por sí mismo, un modelo continuamente sincronizado con los sensores del robot.

Dentro de la arquitectura se distinguen los cuatro grupos principales: percepción, comprensión e interacción, planificación y supervisión, y control de hardware.

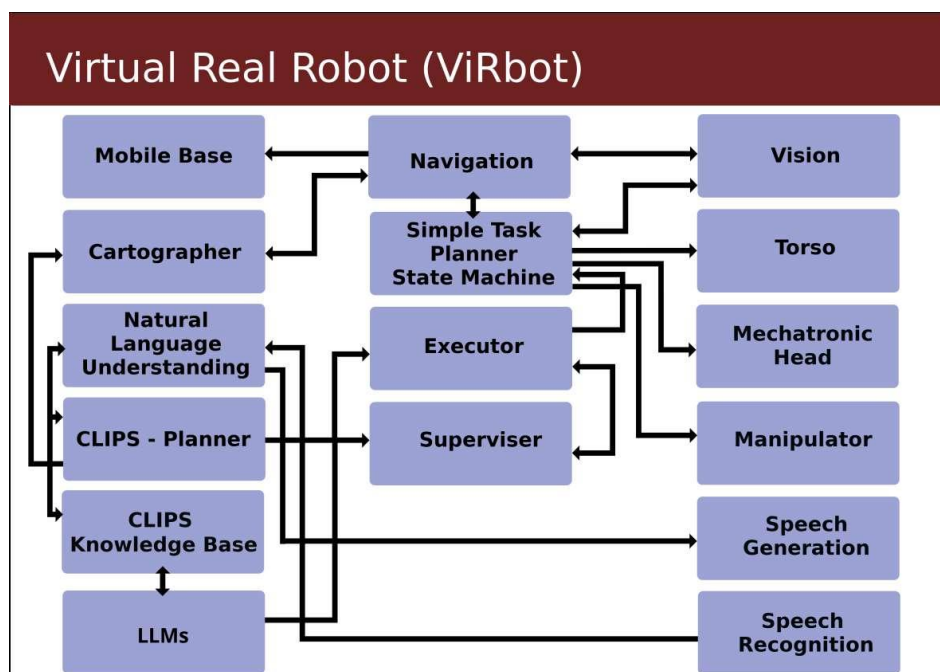


Figura 3.2: Arquitectura Funcional ViRoot.

Módulos de Percepción

Los módulos de percepción permiten al robot obtener información del entorno físico mediante sensores y algoritmos de interpretación.

- **Vision:** Este módulo procesa información proveniente de cámaras y sensores visuales (*nubes de puntos*). Permite la detección y reconocimiento de objetos, personas y características del entorno. La información visual es utilizada principalmente

por el sistema de navegación y por el planificador de tareas para tomar decisiones sobre la interacción con el entorno.

- **Cartographer:** El módulo Cartographer implementa algoritmos de **SLAM** (*Simultaneous Localization and Mapping*) para construir mapas del entorno y estimar la posición del robot dentro de ellos. Este módulo proporciona información fundamental para la navegación autónoma.
- **Speech Recognition:** Este módulo procesa las señales de audio capturadas por los micrófonos del robot y las convierte en texto. El resultado es utilizado por el sistema de *Natural Language Understanding* para interpretar instrucciones del usuario.

Módulos de Comprensión e Interacción

Estos módulos permiten que el robot interactúe con los humanos mediante lenguaje natural.

- **Natural Language Understanding (NLU):** Este componente interpreta las instrucciones del usuario en lenguaje natural. Utiliza modelos lingüísticos y reglas semánticas para transformar frases en representaciones estructuradas que puedan ser utilizadas por el sistema de planificación.
- **LLMs:** Los Large Language Models (LLMs) se emplean como complemento para mejorar la interpretación semántica de las instrucciones. Estos modelos permiten generar representaciones simbólicas o estructuras intermedias que posteriormente son procesadas por el sistema de planificación basado en reglas.
- **Speech Generation:** Este módulo permite al robot generar respuestas habladas. Convierte texto generado por el sistema en señales de voz, facilitando la interacción natural con los usuarios.

Módulos de Planificación y Gestión del Conocimiento

Estos módulos constituyen el núcleo de la toma de decisiones del sistema.

- **CLIPS Knowledge Base:** La base de conocimiento implementada en CLIPS almacena hechos y reglas que representan el conocimiento del entorno, los objetos, las tareas y las capacidades del robot. Esta base de conocimiento se actualiza continuamente conforme el robot percibe el entorno.
- **CLIPS Planner:** Este módulo utiliza reglas de producción para generar planes de acción a partir de objetivos definidos. El planificador determina qué acciones

deben ejecutarse y en qué orden para alcanzar una meta específica.

- **Simple Task Planner State Machine:** Este componente implementa una máquina de estados que coordina la ejecución de tareas simples. Funciona como un nivel intermedio entre la planificación simbólica y la ejecución de acciones físicas.
- **Supervisor:** El supervisor monitorea la ejecución de los planes y gestiona errores o situaciones inesperadas. También se encarga de coordinar la interacción entre los distintos módulos del sistema.
- **Executor:** El ejecutor recibe los planes generados por el planificador y los convierte en comandos concretos para los diferentes actuadores del robot.

Módulos de Control del Robot

Estos módulos controlan directamente los sistemas físicos del robot.

- **Navigation:** El sistema de navegación calcula trayectorias y controla el desplazamiento del robot utilizando información del mapa y de los sensores.
- **Mobile Base:** Este módulo controla la base móvil del robot, permitiendo su desplazamiento en el entorno.
- **Manipulator:** Controla el brazo robótico utilizado para manipular objetos en el entorno.
- **Torso:** Permite el movimiento vertical del cuerpo del robot para alcanzar objetos a diferentes alturas.
- **Mechatronic Head:** Controla los movimientos de la cabeza del robot, permitiendo orientar sensores como cámaras y micrófonos hacia puntos de interés.

3.1. Arquitectura de un Sistema de Planeación de Acciones Jerárquico

La planificación de acciones aborda el problema de generar secuencias de pasos u operaciones que permitan a un robot alcanzar objetivos de alto nivel en entornos parcialmente observables y dinámicos (2). Entre los paradigmas más consolidados para abordar este desafío se encuentra la planificación jerárquica, que descompone tareas complejas en subtareas más simples (22). Por ejemplo ("servir la cena") se descomponen en tareas intermedias ("ir a la cocina", "tomar platos", "colocar en mesa") y finalmente

en acciones primitivas ejecutables por el robot. Este enfoque reduce significativamente la complejidad computacional del problema de planificación al explotar el conocimiento experto sobre cómo descomponer tareas, en contraste con los planificadores clásicos de primer principio que exploran espacios de estados sin guía estructural (23).

3.1.1. Fundamentos de la Planificación Jerárquica: HTN

El método más influyente de planificación jerárquica es el paradigma de **Redes de Tareas Jerárquicas** (*Hierarchical Task Networks, HTN*) (24). En HTN, el problema de planificación no se define mediante un estado objetivo, sino mediante una tarea inicial (task network) que representa el objetivo de alto nivel a alcanzar (22). El conocimiento del dominio se codifica mediante dos elementos fundamentales:

- **Operadores:** Describen acciones primitivas ejecutables directamente por el robot, especificando sus precondiciones y efectos sobre el estado del mundo (25). Son análogos a las acciones en **STRIPS** (*Stanford Research Institute Problem Solver*), el primer planificador clásico ampliamente adoptado (26).
- **Métodos :** Constituyen el núcleo de la jerarquía. Cada método especifica cómo descomponer una tarea compuesta (abstracta) en una red de subtareas, que pueden ser a su vez compuestas o primitivas (27). Los métodos incluyen precondiciones que determinan cuándo es aplicable una determinada descomposición, permitiendo que el planificador elija entre múltiples formas de realizar una misma tarea según el contexto (28).

El proceso de planificación HTN funciona mediante la descomposición recursiva: partiendo de la tarea inicial, el planificador selecciona métodos aplicables cuyas precondiciones se satisfacen en el estado actual, y reemplaza las tareas compuestas por sus subtareas, generando progresivamente una red completamente expandida en acciones primitivas (?). Cuando se alcanza una secuencia de acciones primitivas cuyas precondiciones pueden encadenarse consistentemente desde el estado inicial, se ha encontrado un plan solución.

Una ventaja distintiva de HTN es su capacidad para generar planes explicables y alineados con el conocimiento experto, ya que la jerarquía de métodos refleja cómo un diseñador humano concebiría la realización de tareas complejas (27).

3.1.2. El Planificador HATP (Hierarchical Agent-Based Task Planner)

Dentro de la familia de planificadores HTN orientados a robótica, destaca HATP (Hierarchical Agent-based Task Planner) , desarrollado en LAAS-CNRS para entornos de interacción humano-robot (27, 29). HATP extiende el paradigma HTN tradicional con características específicamente diseñadas para robótica de servicio:

- **Tratamiento de agentes como entidades de primera clase:** HATP distingue explícitamente entre diferentes agentes (robots, humanos) en la representación del dominio, permitiendo asignar tareas a distintos ejecutores y generar planes multi-agente sincronizados (30).
- **Reglas sociales (social rules):** Incorpora mecanismos para especificar restricciones sobre comportamientos aceptables en contextos de interacción humano-robot, como límites de tiempo de espera o secuencias de acciones que deben evitarse por razones de comodidad o seguridad (27).
- **Integración con razonamiento geométrico:** HATP puede intercalar planificación simbólica con validación geométrica mediante planificadores de movimiento (ej. Move3D), verificando en tiempo de planificación si las acciones consideradas son factibles en el mundo físico modelado con detalle 3D (29).

3.1.3. Taxonomía de Arquitecturas Jerárquicas en Robótica

Más allá de HTN, la literatura identifica diversas aproximaciones a la planificación jerárquica en robótica:

- **Planificación Jerárquica con Lógica Temporal (HTL):** Una línea reciente de investigación extiende la planificación jerárquica al dominio de especificaciones de lógica temporal proponiendo un framework de descomposición jerárquica basado en **LTL** (*Linear Temporal Logic*) jerárquico (31), donde cada especificación de alto nivel se descompone en subtareas atómicas, infiriendo relaciones temporales entre ellas para construir una red de tareas. Este enfoque, validado experimentalmente en dominios de navegación y manipulación, permite manejar especificaciones complejas que resultarían inmanejables con una única fórmula LTL monolítica.
- **Arquitecturas de Tres Capas (Three-Layer Architectures):** Un patrón arquitectónico ampliamente adoptado en robótica autónoma es la separación en

capas deliberativa, de ejecución y reactiva (32). En el contexto de rehabilitación robótica, esta estructura se manifiesta como:

- **Capa física:** Maneja la interacción directa con el entorno mediante sensores y actuadores.
 - **Capa reactiva:** Ajusta en tiempo real parámetros de control (ej. asistencia, dificultad) basándose en retroalimentación sensorial.
 - **Capa deliberativa:** Utiliza planificadores como HTN o planificación basada en línea temporal para generar estrategias de intervención personalizadas a largo plazo (33).
- **Planificación Jerárquica con Representaciones Ontológicas:** Investigaciones recientes exploran la integración de ontologías para enriquecer la representación del conocimiento en planificadores jerárquicos (34). Las ontologías permiten modelar relaciones semánticas entre objetos, agentes y tareas, facilitando el razonamiento sobre contextos no explícitamente programados y mejorando la adaptabilidad del planificador a situaciones novedosas.

3.1.4. Relevancia para el Sistema Propuesto

La arquitectura de planificación jerárquica presentada constituye el fundamento teórico sobre el cual se construye el sistema híbrido CLIPS-ChatGPT/Qwen propuesto en esta tesis. Específicamente:

- El motor CLIPS implementa un sistema de producción que, aunque no sigue estrictamente el formalismo HTN, puede organizarse jerárquicamente mediante reglas de prioridad y descomposición secuencial, como se detalla en la sección 4.1.1 (9).
- La integración con ChatGPT/Qwen para planificación dual (sección 4.3) puede entenderse como una extensión de las arquitecturas híbridas que combinan planificadores simbólicos (como HATP) con módulos de razonamiento geométrico o semántico, pero sustituyendo estos últimos por modelos de lenguaje de gran escala con capacidad de generar planes contextualmente adaptativos (6).
- La arquitectura ViRoot del robot Justina (Figura 3.1) sigue explícitamente una organización jerárquica en capas de entradas, planificación, gestión de conocimiento y ejecución, alineándose con los principios de las arquitecturas de tres capas discutidas (9).

Esta base teórica proporciona el marco necesario para comprender cómo la combina-

ción de razonamiento simbólico jerárquico y procesamiento flexible de lenguaje natural puede superar las limitaciones de ambos paradigmas por separado, constituyendo la hipótesis central de esta investigación.

3.2. Fundamentos de los Sistemas de Producción (Rule-Based Systems)

Los sistemas de producción, también conocidos como sistemas basados en reglas (*Rule-Based Systems*, *RBS*), constituyen uno de los paradigmas más implementados en inteligencia artificial para la representación del conocimiento y el razonamiento automatizado (35). Su fundamento teórico reside en la representación del conocimiento mediante reglas del tipo if-then (*condición-acción*), que operan sobre una base de hechos que describe el estado actual del mundo (36). En robótica, estos sistemas han demostrado particular utilidad para la planificación de acciones en entornos estructurados, donde la previsibilidad y la capacidad de explicación son requisitos fundamentales (35).

3.2.1. Estructura y Componentes de un Sistema de Producción

El sistema se compone de tres elementos fundamentales que interactúan de manera cíclica (35):

1. **Base de Hechos:** Almacena la información relevante sobre el estado actual del entorno y del propio robot. Estos hechos representan afirmaciones sobre el mundo, como la ubicación de objetos, el estado de los actuadores o las percepciones sensoriales. En el contexto del robot Justina, la memoria de trabajo contendría hechos como (*ubicacion robot cocina*) o (*objeto vaso mesa*).
2. **Base de Reglas:** Contiene el conjunto de reglas de producción que codifican el conocimiento del dominio. Cada regla presenta la forma general (36):

$$(condition1)(condition2)...(conditionm) \Rightarrow (action1)(action2)...(actionn)$$

La parte izquierda especifica las condiciones que deben satisfacerse para que la

regla sea aplicable, mientras que la parte derecha define las acciones a ejecutar, que típicamente modifican la memoria de trabajo o producen efectos externos (35).

3. **Motor de Inferencia:** Constituye el mecanismo de control que gobierna la aplicación de las reglas. Es responsable de determinar qué reglas son aplicables en cada momento, seleccionando una entre las posibles, y ejecutando sus acciones (35). El motor de inferencia implementa el ciclo fundamental de operación, es decir, el ciclo reconocer-actuar (*recognize-act cycle*) (2).

3.2.2. El Ciclo Reconocer-Actuar

Este ciclo constituye el corazón operativo de todo sistema basado en reglas. El proceso iterativo se desarrolla en las siguientes fases (2) (35):

1. **Reconocimiento:** El motor de inferencia compara el contenido actual de la memoria de trabajo con las condiciones de todas las reglas en la base de conocimiento. Aquellas reglas cuyas condiciones se satisfacen completamente con los hechos existentes se consideran "*activadas*" y pasan a formar parte del conjunto de conflictos (2).
2. **Resolución de Conflictos:** Cuando múltiples reglas resultan aplicables simultáneamente, el motor debe seleccionar una para su ejecución. Esta decisión se realiza mediante estrategias de resolución de conflictos, que pueden basarse en prioridades explícitas de las reglas, especificidad de las condiciones, orden de activación, o políticas específicas del dominio (35).
3. **Actuación:** La regla seleccionada se ejecuta, realizando las acciones especificadas en su parte derecha. Estas acciones típicamente modifican la memoria de trabajo (añadiendo, eliminando o actualizando hechos) y pueden tener efectos secundarios sobre el entorno, y el robot (2) (35).
4. **Terminación:** El ciclo se repite hasta que se cumple un criterio de terminación, que puede ser la ausencia de reglas aplicables, la consecución del objetivo deseado, o una condición explícita de parada (35).

3.2.3. El Algoritmo RETE

Una implementación ingenua del ciclo *reconocer-actuar* comprobaría cada regla contra todos los hechos en cada ciclo, resultando en una complejidad computacional prohi-

bitiva incluso para sistemas de tamaño moderado (37). Para abordar esta limitación, Charles Forgy desarrolló el **algoritmo RETE** durante su doctorado en *Carnegie Mellon University* (1979), cuya publicación apareció en *Artificial Intelligence* en 1982. El nombre "*RETE*" proviene del latín y significa "*red*", reflejando la estructura de red que constituye su esencia (37).

Principios Fundamentales del Algoritmo RETE

El algoritmo RETE optimiza el proceso de emparejamiento *patrón-objeto* mediante dos estrategias clave (37):

1. **Almacenamiento de Estado: RETE** mantiene en memoria los resultados de emparejamientos previos, de modo que solo es necesario procesar los cambios incrementales en la memoria de trabajo, en lugar de reevaluar todas las reglas desde cero en cada ciclo.
2. **Compartición de subcálculos:** Cuando múltiples reglas comparten condiciones comunes, **RETE** evalúa dichas condiciones una sola vez y reutiliza los resultados para todas las reglas que las contienen.

Estos principios se implementan mediante la construcción de una red de nodos (*RETE network*) que representa las condiciones de todas las reglas del sistema (37) (3).

Estructura de la Red RETE

La red RETE se organiza jerárquicamente desde el nodo raíz hasta las hojas que representan reglas completas (37):

- **Nodo Raíz:** Punto de entrada por el que fluyen todos los hechos introducidos en el sistema (3).
- **Nodos Alfa:** Realizan pruebas simples sobre hechos individuales, como comprobaciones de tipo o igualdad de atributos. Cada nodo alfa corresponde a un patrón atómico dentro de las condiciones de las reglas (38). Estos nodos operan de manera independiente y almacenan en su memoria los hechos que satisfacen su prueba (37).
- **Nodos Beta:** Implementan operaciones de unión entre conjuntos de hechos procedentes de diferentes caminos en la red. Cada nodo beta combina los resultados de dos nodos anteriores (típicamente uno alfa y otro beta) verificando que las variables compartidas tengan vinculaciones consistentes (39) (38). La memoria

de los nodos beta almacena los pares o tuplas que han superado la condición de unión (37).

- **Nodos Terminales:** Representan reglas completas. Cuando un token alcanza un nodo terminal, significa que se ha encontrado una combinación de hechos que satisface todas las condiciones de la regla correspondiente, quedando esta activada para su ejecución (37) (3).

Funcionamiento Incremental

El proceso de evaluación en **RETE** es incremental (39). Cuando se añade, modifica o elimina un hecho en la memoria de trabajo:

1. El hecho ingresa por el nodo raíz y se propaga hacia abajo a través de los nodos alfa cuyas pruebas supera (37).
2. Los nodos alfa actualizan sus memorias y propagan los cambios a los nodos beta conectados.
3. Los nodos beta evalúan las nuevas combinaciones posibles con los tokens almacenados en sus memorias izquierda y derecha, actualizando sus propias memorias y propagando los resultados (39) (3).
4. Finalmente, los cambios alcanzan los nodos terminales, actualizando dinámicamente el conjunto de reglas activadas (37).

Este enfoque incremental reduce drásticamente el trabajo computacional, ya que solo se procesan los cambios efectivos en lugar de reevaluar todo el sistema en cada ciclo. La complejidad temporal del algoritmo **RETE** es, en promedio, independiente del número total de reglas en el sistema, dependiendo principalmente del número de hechos y de la estructura de las condiciones (40).

3.2.4. Implementación de RETE en CLIPS

CLIPS implementa una versión optimizada del algoritmo RETE adaptada a sus necesidades específicas como sistema de producción de propósito general .

Estructura de Memoria en CLIPS

CLIPS mantiene dos tipos principales de memoria (41):

- **Memoria de Trabajo:** Almacena los hechos que representan el estado actual del sistema. Cada hecho puede ser simple (ordenado) o estructurado mediante

plantillas (deftemplate).

- **Memoria de Hechos:** Una estructura interna que permite el acceso eficiente a los hechos durante el proceso de emparejamiento.

El Motor de Inferencia de CLIPS

Se implementa el ciclo *reconocer-actuar* completo, utilizando la red **RETE** para la fase de reconocimiento (41):

1. **Fase de Reconocimiento:** Los cambios en la memoria de trabajo se propagan a través de la red **RETE** precompilada a partir de las reglas del sistema. La red mantiene información sobre qué condiciones se satisfacen parcial o completamente.
2. **Fase de Resolución de Conflictos:** CLIPS ofrece múltiples estrategias de resolución de conflictos, incluyendo prioridad (saliency), profundidad (depth-first), amplitud (breadth-first), y estrategias basadas en la especificidad de las reglas (41).
3. **Fase de Actuación:** La regla seleccionada ejecuta sus acciones, que pueden incluir la modificación de la memoria de trabajo, la ejecución de funciones externas.

Consideraciones de Rendimiento

Estudios sobre implementaciones de CLIPS en entornos empotrados han identificado cinco parámetros clave que afectan el rendimiento del motor de inferencia (1):

1. El número de objetos en la memoria de trabajo.
2. La complejidad estructural de los objetos y las reglas.
3. El número de reglas que comparten patrones de emparejamiento comunes.
4. El número de patrones por regla.
5. El número de objetos vinculados a cada patrón.

En aplicaciones de tiempo real el tiempo de respuesta del motor de inferencia suele ser adecuado, aunque los requisitos de memoria pueden ser significativos dependiendo de la complejidad de la red **RETE** (1). Esta característica hace que CLIPS sea particularmente eficiente para plataformas robóticas con restricciones computacionales.

3.2.5. Ventajas y Limitaciones de los Sistemas de Producción

Los sistemas basados en reglas presentan ventajas significativas que los hacen útiles en robótica (35):

■ **Ventajas:**

- **Modularidad:** Las reglas pueden añadirse, modificarse o eliminarse independientemente, facilitando el mantenimiento y la evolución del sistema.
- **Explicabilidad:** El razonamiento puede rastrearse y explicarse mediante la inspección de las reglas activadas, lo que es crucial para la auditoría y la confianza en sistemas críticos.
- **Naturalidad:** Las reglas expresadas en formato *if-then* se aproximan a la forma en que los humanos describen procedimientos y conocimientos.
- **Separación conocimiento-control:** El conocimiento del dominio (*reglas*) está claramente separado del mecanismo de control (*motor de inferencia*), permitiendo modificaciones sin afectar la arquitectura subyacente.

■ **Limitaciones:**

- **Dificultad de análisis:** El flujo de control puede ser difícil de predecir cuando múltiples reglas interactúan de maneras complejas.
- **Ausencia de aprendizaje:** Los sistemas clásicos de producción no incorporan capacidades de aprendizaje automático, requiriendo que todo el conocimiento sea explícitamente programado.
- **Ineficiencia potencial:** Aunque **RETE** optimiza el emparejamiento, sistemas con miles de reglas pueden presentar problemas de rendimiento.
- **Gestión de conflictos:** La resolución de conflictos requiere diseños cuidadosos para evitar comportamientos indeseados.

Estas limitaciones, particularmente la rigidez semántica y la ausencia de aprendizaje, motivan la integración con modelos de lenguaje grande propuesta en esta tesis, buscando combinar la robustez y transparencia de los sistemas basados en reglas con la flexibilidad y capacidad adaptativa de los enfoques subsimbólicos modernos.

3.3. Introducción a los Modelos de Lenguaje Grande (LLMs) y ChatGPT/Qwen

Los Modelos de Lenguaje Grande (*Large Language Models*, **LLMs**) han emergido como una de las tecnologías más transformadoras en el campo de la inteligencia artificial, revolucionando la manera en que las máquinas procesan y generan lenguaje natural (6). Su aplicación en robótica de servicio abre nuevas posibilidades para la interacción humano-robot, permitiendo que usuarios no expertos comuniquen sus in-

tenciones mediante comandos verbales complejos sin necesidad de conocer lenguajes de programación estructurados (42). Se exploran los fundamentos de la arquitectura de los LLMs, centrándose en el mecanismo de atención y la arquitectura Transformer, para posteriormente contrastar dos modelos representativos: ChatGPT (modelo propietario) y Qwen2.5-0.5B (modelo de código abierto), evaluando su idoneidad para entornos robóticos con recursos computacionales limitados.

3.3.1. Transformers y Mecanismo de Atención

Limitaciones de los Modelos Secuenciales Previos

Antes de la introducción de la arquitectura Transformer en 2017, el procesamiento de lenguaje natural estaba dominado por redes neuronales recurrentes (*RNN*) y sus variantes, como **LSTM** (*Long Short-Term Memory*) y **GRU** (*Gated Recurrent Units*) (2). Estos modelos procesan el texto de manera secuencial, token por token, lo que presenta dos limitaciones fundamentales (5):

1. **Dificultad para el paralelismo:** La naturaleza secuencial de las **RNN** impide el aprovechamiento eficiente de hardware paralelo como GPUs, resultando en tiempos de entrenamiento prolongados (38).
2. **Problema de dependencias de largo alcance:** Aunque **LSTM** y **GRU** mitigan parcialmente el problema del gradiente descendente, la información sobre palabras distantes tiende a diluirse a medida que la secuencia se alarga, dificultando el modelado de relaciones semánticas entre elementos lejanos (43).

La Arquitectura Transformer

El artículo «Attention Is All You Need», publicado por investigadores de Google en 2017, introdujo la arquitectura Transformer, que prescinde por completo de recurrencia y convoluciones, basándose únicamente en mecanismos de atención (5). Esta arquitectura se ha convertido en el fundamento de todos los **LLMs** modernos, incluyendo **GPT**, **BERT**, y sus derivados (6).

La arquitectura Transformer sigue una estructura **codificador-decodificador** (*encoder-decoder*) con las siguientes características principales (5):

- **Codificador (Encoder):** Compuesto por una pila de $N = 6$ capas idénticas. Cada capa contiene dos subcapas: un mecanismo de autoatención multi-cabeza y una red feed-forward posicional totalmente conectada. Alrededor de cada subcapa

se emplean conexiones residuales seguidas de normalización por capas.

- **Decodificador (Decoder):** Similar al codificador, pero con una tercera subcapa que realiza atención cruzada sobre la salida del codificador. Además, incorpora un enmascaramiento para evitar que las posiciones futuras influyan en la predicción actual, preservando la propiedad autorregresiva del modelo.

El Mecanismo de Atención

El núcleo de la arquitectura Transformer es el mecanismo de atención, que permite al modelo ponderar dinámicamente la importancia de diferentes elementos de la secuencia de entrada al generar cada representación de salida (5). Formalmente, la atención puede describirse como una función que mapea una consulta y un conjunto de pares clave-valor hacia una salida (44).

Atención Escalada por Producto Punto: La forma de atención utilizada en Transformer se define mediante la siguiente ecuación (5):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (3.1)$$

Donde:

- Q (Query) es la representación del token actual que "pregunta" por información relevante.
- K (Key) representa las "etiquetas" de todos los tokens que pueden responder a la consulta.
- V (Value) contiene la información real de cada token que será agregada.
- d_k es la dimensión de las claves, y la división por $\sqrt{d_k}$ actúa como factor de escalamiento para evitar que los productos punto crezcan excesivamente, lo que llevaría a gradientes extremadamente pequeños en la función softmax (5).

Interpretación intuitiva del mecanismo de atención: Una analogía útil para comprender este mecanismo es la de una biblioteca (45). La biblioteca (*Source*) contiene muchos libros (*Value*), cada uno con un código de clasificación (*Key*). Cuando un usuario desea información sobre un tema específico (*Query*), el sistema busca los códigos más relevantes y recupera los libros correspondientes, prestando más atención (*pesos mayores*) a aquellos que mejor coinciden con la consulta.

Autoatención: Cuando Q, K y V se generan a partir de la misma secuencia de

entrada, el mecanismo se denomina autoatención. Esto permite que cada token de la secuencia "atienda" a todos los demás tokens, capturando relaciones contextuales globales independientemente de la distancia entre ellos (5).

Atención Multi-Cabeza: En lugar de realizar una única función de atención, Transformer proyecta Q , K y V h veces con proyecciones lineales aprendidas diferentes, permitiendo al modelo atender conjuntamente a información procedente de distintos subespacios representacionales (5):

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (3.2)$$

$$\text{donde } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Codificación Posicional (Positional Encoding): Dado que la autoatención es inherentemente invariante al orden de los tokens (*permutar la secuencia produce el mismo conjunto de atenciones*), es necesario inyectar información sobre la posición relativa o absoluta de los tokens. Transformer utiliza codificaciones posicionales sinusoidales (5):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3.3)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3.4)$$

Modelos más recientes han adoptado variantes como **RoPE** (*Rotary Positional Embeddings*), que codifica posiciones relativas mediante rotaciones de los vectores QyK , mejorando la extrapolación a longitudes de secuencia superiores a las vistas durante el entrenamiento (46)(47).

Evolución hacia Modelos de Lenguaje Grande

A partir de la arquitectura Transformer, la investigación en **NLP** se bifurcó en dos direcciones principales (46):

1. **Modelos tipo codificador (Encoder-only):** Como **BERT** (*Bidirectional Encoder Representations from Transformers*), que aprovecha la bidireccionalidad del codificador para tareas de comprensión del lenguaje, como clasificación o reconocimiento de entidades (45). Estos modelos se pre-entrenan con objetivos como el modelado de lenguaje enmascarado (*Masked Language Modeling*, **MLM**).
2. **Modelos tipo decodificador (Decoder-only):** Como la serie **GPT** (*Generative Pre-trained Transformer*), que utilizan únicamente la parte decodificadora

con enmascaramiento causal, optimizados para generación de texto autorregresiva (46). Estos modelos se pre-entrenan con el objetivo de predecir el siguiente token t (**Causal Language Modeling, CLM**). Los **LLMs** actuales, incluyendo **ChatGPT** y **Qwen**, pertenecen predominantemente a esta categoría.

3.3.2. ChatGPT: Modelo Propietario para Interacción Conversacional

ChatGPT, desarrollado por **OpenAI**, es un modelo de lenguaje grande basado en la arquitectura **GPT** descrita anteriormente, específicamente optimizado para tareas de diálogo mediante técnicas de ajuste fino con aprendizaje por refuerzo a partir de retroalimentación humana (**RLHF**) (6). Su integración en robótica ha demostrado ser prometedora para interpretar comandos complejos en lenguaje natural y generar planes de acción contextualmente apropiados.

■ Características principales de ChatGPT:

- **Arquitectura:** Basada en Transformer con decodificador causales, escala a cientos de miles de millones de parámetros en sus versiones más avanzadas (6). Aunque los detalles arquitectónicos precisos no son públicos, sigue los principios fundamentales establecidos.
- **Capacidades conversacionales:** Esta optimizado para mantener diálogos coherentes y contextualmente relevantes, manejar ambigüedades y solicitar aclaraciones cuando es necesario (42).
- **Acceso mediante API:** OpenAI proporciona interfaces de programación que permiten integrar **ChatGPT** en sistemas robóticos mediante llamadas a servicios web, facilitando su adopción sin necesidad de infraestructura local costosa (48).

■ Ventajas para robótica:

- **Alta calidad de comprensión:** Demuestra una capacidad superior para interpretar comandos complejos y ambiguos, con tasas de éxito del 100
- **Flexibilidad semántica:** Puede generar planes adaptativos para situaciones novedosas que no están explícitamente programadas en sistemas basados en reglas (6).
- **Reducción de latencia:** Investigaciones recientes muestran que la integración directa de **ChatGPT** con **ROS 2** puede reducir la latencia de comuni-

cación en un 7.01 % en promedio, facilitando operaciones robóticas en tiempo real (48).

■ **Limitaciones en entornos robóticos:**

- **Dependencia de conectividad:** Requiere acceso a internet para consultar la API de **OpenAI**, lo que puede ser problemático en entornos con conectividad limitada o requisitos de seguridad que impidan la transmisión de datos a la nube (?).
- **Costos operativos:** El uso de API propietarias implica costos por consulta que pueden ser prohibitivos en despliegues a gran escala o con alta frecuencia de interacción (49).
- **Latencia variable:** Los tiempos de respuesta dependen de la calidad de la conexión y la carga de los servidores de **OpenAI**, lo que puede introducir variabilidad indeseable en sistemas de tiempo real (48).
- **Privacidad de datos:** El envío de comandos de voz y datos contextuales a servidores externos puede plantear problemas de confidencialidad en aplicaciones sensibles (49).

3.3.3. Qwen2.5-0.5B: Modelo Open-Source para Cómputo

Qwen (*QwenLM*) es una familia de modelos de lenguaje grande desarrollada por **Alibaba**, con versiones que abarcan desde modelos masivos hasta variantes extremadamente compactas diseñadas para despliegue en entornos con recursos limitados (?). **Qwen2.5-0.5B-Instruct**, la versión con 0.5 mil millones de parámetros, representa un punto óptimo entre capacidad lingüística y eficiencia computacional, siendo particularmente relevante para robótica de servicio.

■ **Características principales de Qwen2.5-0.5B:**

Característica	Descripción
Parámetros	0.5 mil millones (aproximadamente 1 GB en pesos del modelo) (?)
Arquitectura	Transformer con decodificador causal, similar a GPT (?)
Soporte de inferencia	CPU nativo, memoria < 2 GB, sin necesidad de GPU (?) (49)
Idiomas	Optimizado para chino, con capacidades funcionales en inglés (?)
Ajuste	Instrucción fine-tuning (versión Instruct) para diálogo y seguimiento de comandos (49)
Formato de despliegue	Disponible en GGUF (llama.cpp), ONNX, y formatos nativos de PyTorch (?)

■ **Ventajas para robótica:**

- **Ejecución local y privada:** Todo el procesamiento ocurre en el dispositivo, eliminando dependencias de red y garantizando la privacidad de los datos (49).
- **Bajos requisitos computacionales:** Puede ejecutarse en procesadores ARM (como Raspberry Pi 4/5) y x86 de gama baja, con consumo de memoria inferior a 2 GB tras cuantización INT8 (?). Tiempos de inferencia típicos: 300-800 ms para primer token en CPU de 4 núcleos (?).
- **Costo cero por consulta:** Al ser código abierto y ejecutarse localmente, no hay costos asociados al número de interacciones.
- **Personalización:** Posibilidad de fine-tuning con datos específicos del dominio (*por ejemplo, vocabulario de un laboratorio o nombres de objetos domésticos*) sin dependencia de proveedores externos (49).
- **Integración con ROS 2:** Experiencias documentadas muestran la viabilidad de integrar **Qwen2.5-0.5B** con **ROS 2** para aplicaciones como guías inteligentes en entornos educativos, donde el modelo procesa consultas en lenguaje natural y genera respuestas que se traducen en comandos de navegación (49).

■ **Limitaciones:**

- **Capacidad lingüística reducida:** Con solo 0.5B parámetros, su comprensión y generación son inferiores a modelos como ChatGPT, especialmente

para comandos muy complejos o ambiguos (49). Sin embargo, para dominios restringidos (vocabulario de un hogar o laboratorio), el rendimiento puede ser suficiente tras personalización.

- **Requiere optimización:** Para alcanzar latencias aceptables en CPU, es necesario aplicar técnicas de cuantización (INT8, INT4) y utilizar motores de inferencia eficientes como llama.cpp u ONNX Runtime (?).
- **Menor capacidad de diálogo:** La gestión de contextos largos y la coherencia en diálogos extendidos son inferiores a las de modelos con más parámetros (47).

3.3.4. Comparativa y Selección para Robótica de Servicio

La Tabla 3.1 resume las principales diferencias entre **ChatGPT** (*representante de modelos propietarios*) y **Qwen2.5-0.5B** (*representante de modelos open-source ligeros*), proporcionando criterios para la selección según las necesidades de la aplicación robótica.

En el contexto de esta tesis, la selección entre ambos modelos responde a una estrategia complementaria (9):

- **Qwen2.5-0.5B** se emplea como módulo de traducción de lenguaje natural a hechos CLIPS directamente en el robot, aprovechando su capacidad para ejecutarse localmente con latencias predecibles y sin dependencia de conectividad. Esto es particularmente útil para tareas rutinarias donde el vocabulario está acotado al entorno doméstico.
- **ChatGPT** se utiliza en el módulo de planificación dual o híbrido, como interprete de lenguaje natural o generador de planes alternativos para situaciones complejas o novedosas, cuando el sistema basado en reglas no puede resolver la tarea. En estos casos, la latencia adicional se justifica por la necesidad de flexibilidad semántica.

Tabla 3.1: Comparativa entre ChatGPT y Qwen2.5-0.5B para robótica de servicio

Aspecto	ChatGPT	Qwen2.5-0.5B
Parámetros	> 100B (estimado)	0.5B
Ejecución	Nube (API)	Local (edge)
Latencia típica	500-2000 ms (variable)	300-800 ms (consistente) (?)
Dependencia de red	Obligatoria	Ninguna
Privacidad	Datos enviados a terceros	Datos locales
Costo por consulta	Sí (por token)	No
Personalización	Limitada (fine-tuning no disponible)	Completa (fine-tuning local)
Calidad lingüística	Muy alta	Moderada (suficiente para dominios restringidos) (49)
Consumo energético	Alto (servidores)	Bajo (CPU)
Idoneidad	Tareas complejas, investigación, prototipado	Despliegue en producción, dispositivos con recursos limitados, privacidad crítica

Esta arquitectura híbrida capitaliza las ventajas de ambos paradigmas: la eficiencia y privacidad de la ejecución local con **Qwen** para tareas estándar, y la potencia lingüística de **ChatGPT** para casos excepcionales, manteniendo siempre la supervisión del sistema basado en reglas como garantía de seguridad (6).

3.4. Integración de Traductores de Lenguaje Natural con Sistemas Basados en Reglas

Esta integración está enfocada en la arquitectura que busca combinar la robustez y transparencia del razonamiento simbólico con la flexibilidad y accesibilidad de la interacción en lenguaje humano. En el contexto de la robótica de servicio resulta fundamental permitir que usuarios no expertos puedan comunicarse con el robot de manera intuitiva, utilizando el lenguaje natural, mientras que el sistema subyacente mantiene la predictibilidad y seguridad propias de los motores de reglas. Se aborda la utilidad de dicha integración, describiendo su funcionamiento técnico en el caso específico de CLIPS, y presenta una descripción comparativa de las tres tecnologías involucradas:

CLIPS, ChatGPT y Qwen2.5-0.5B.

3.4.1. Utilidad y Función de la Integración

La principal utilidad de integrar traductores de lenguaje natural con sistemas basados en reglas radica en la ampliación de la accesibilidad del sistema robótico. Tradicionalmente, la programación y operación de robots basados en reglas requiere que el usuario conozca la sintaxis formal de hechos y reglas, o que utilice interfaces gráficas predefinidas que limitan la expresividad de las instrucciones (3). Al incorporar un módulo de procesamiento de lenguaje natural, el robot puede interpretar comandos expresados en lenguaje cotidiano, tales como "tráeme el vaso rojo que está en la mesa de la cocina", y transformarlos en representaciones estructuradas que el motor de reglas puede procesar (42).

Esta integración cumple varias funciones esenciales:

1. **Interfaz intuitiva para usuarios no técnicos:** Elimina la necesidad de aprender lenguajes de programación o secuencias de comandos predefinidas, permitiendo que personas sin formación en robótica puedan interactuar con el robot de manera natural (39).
2. **Traducción de ambigüedad a precisión:** El lenguaje natural es inherentemente ambiguo; una misma frase puede tener múltiples interpretaciones. El módulo de traducción debe resolver estas ambigüedades mediante análisis semántico y contextual, generando hechos unívocos que el sistema basado en reglas pueda evaluar sin ambigüedad (44). Técnicas como la desambiguación del sentido de las palabras (*word sense disambiguation*) y el reconocimiento de entidades nombradas son fundamentales en este proceso (?).
3. **Enriquecimiento del conocimiento del dominio:** Los comandos en lenguaje natural pueden contener información implícita sobre el entorno, los objetos y las intenciones del usuario que no está disponible en la base de conocimiento inicial. El traductor puede extraer esta información y representarla como nuevos hechos, enriqueciendo dinámicamente la memoria de trabajo del sistema de reglas (36).
4. **Adaptación a contextos culturales y situacionales:** Diferentes usuarios pueden referirse al mismo objeto o acción con términos distintos. Un traductor basado en modelos de lenguaje grande puede capturar estas variaciones lingüísticas y mapearlas a los conceptos canónicos definidos en la base de reglas, mejorando la robustez del sistema en entornos multilingües o multiculturales (45).

En el ámbito de la robótica de servicio doméstico, esta integración resulta particularmente valiosa para aplicaciones de asistencia a personas mayores o con alguna discapacidad, donde la facilidad de uso y la naturalidad de la interacción son requisitos prioritarios.

3.4.2. Funcionamiento de la Integración CLIPS-Lenguaje Natural

La integración entre CLIPS y los modelos de lenguaje natural en el robot Justina sigue un flujo de procesamiento claramente definido, que transforma comandos verbales en hechos estructurados y, finalmente, en planes ejecutables. Este flujo se organiza en torno a servicios y tópicos de ROS 2 que garantizan la comunicación asíncrona y desacoplada entre los módulos (9).

Flujo de Procesamiento

El proceso completo de integración comprende las siguientes etapas:

1. **Captura y reconocimiento de voz:** El robot Justina captura el audio del entorno mediante su micrófono y utiliza un motor de reconocimiento de voz, como Vosk, para transcribir el habla a texto (49). Esta etapa convierte la señal acústica en una cadena de texto que representa el comando del usuario.
2. **Análisis semántico mediante LLM:** El texto resultante se envía al módulo de lenguaje natural, que puede estar basado en ChatGPT (mediante API) o en Qwen2.5-0.5B (ejecución local). Este módulo realiza un análisis semántico profundo para identificar los componentes clave del comando (?):
 - **Acción principal:** "traer", "llevar", "buscar", "colocar".
 - **Objeto meta:** "vaso", "libro", "medicina".
 - **Atributos descriptivos:** "rojo", "pequeño", "de plástico".
 - **Ubicaciones:** ".en la mesa", ".en la cocina", "sobre el estante".
3. **Generación de hechos CLIPS:** El resultado del análisis se transforma en hechos estructurados según la sintaxis de CLIPS. Un hecho típico generado tiene la forma:

(objetivo (accion traer) (objeto vaso) (atributo rojo) (ubicacion mesa-cocina))

Esta representación sigue las plantillas definidas en la base de conocimiento del

sistema, asegurando que los hechos generados sean directamente procesables por el motor de reglas (3).

4. **Inserción en la memoria de trabajo:** Los hechos generados se insertan en la memoria de trabajo de **CLIPS** mediante el servicio **ROS2** correspondiente. La inserción desencadena el ciclo de inferencia del motor de reglas, que evalúa las reglas aplicables y genera un plan de acciones (36).
5. **Ejecución del plan:** El plan resultante, consistente en una secuencia de acciones primitivas, se publica en un tópico **ROS2** para que los módulos de ejecución (navegación, manipulación, etc.) lo traduzcan en comandos concretos para los actuadores del robot (9).

Servicios y Tópicos ROS2

La arquitectura de integración se implementa mediante los siguientes elementos de comunicación en ROS2 (45):

- **Servicio** */natural_to_clips/interpret*:
 - **Entrada:** Cadena de texto con el comando en lenguaje natural.
 - **Procesamiento:** Invoca al modelo de lenguaje (**ChatGPT** o **Qwen**) con un prompt estructurado que incluye el contexto del entorno y ejemplos de traducción esperados.
 - **Salida:** Lista de hechos CLIPS estructurados en formato JSON o similar.
 - **Validación:** Se aplican técnicas de prompting few-shot y validación sintáctica para garantizar que los hechos generados sean correctos (?).
- **Servicio** */clips/assert_fact*:
 - **Entrada:** Hecho individual en formato CLIPS (ej., (objetivo (accion traer (objeto vaso))).
 - **Procesamiento:** Inserta el hecho en la memoria de trabajo de CLIPS y retorna una confirmación.
 - **Manejo de errores:** Verifica que el hecho respete las plantillas definidas; en caso contrario, rechaza la inserción y notifica el error.
- **Tópico** */clips/results*:
 - **Publicación:** El motor CLIPS publica el plan completo generado tras el ciclo de inferencia.
 - **Formato:** Secuencia ordenada de acciones primitivas, cada una con los parámetros necesarios para su ejecución (ej., (goto cocina), (find-object va-

so), (grasp objeto)).

- **Consumidores:** Los módulos de navegación, manipulación y control de actuadores usan esta información para ejecutar las acciones.

Técnicas de Prompting para la Generación de Hechos

La calidad de la traducción de lenguaje natural a hechos CLIPS depende críticamente del diseño de los prompts enviados al modelo de lenguaje, se emplean las siguientes técnicas:

- **Prompts con especificación de tarea:** Se proporciona al modelo una descripción clara de su función: "Eres un asistente que traduce comandos en lenguaje natural a hechos estructurados para un robot doméstico. Los hechos deben seguir el formato (objetivo (accion ¡acción_i!) (objeto ¡objeto_i!) (atributo ¡atributo_i!) (ubicacion ¡ubicación_i!))."
- **Contexto del entorno:** Se incluye información sobre el entorno doméstico (habitaciones disponibles, objetos conocidos, capacidades del robot) para guiar la generación de hechos realistas y ejecutables (?).
- **Ejemplos few-shot:** Se proporcionan varios ejemplos de comandos y sus correspondientes hechos CLIPS, demostrando cómo manejar variaciones lingüísticas y casos ambiguos (?).
- **Validación y retroalimentación:** Cuando el modelo genera hechos sintácticamente incorrectos o incompletos, se envía un mensaje de error detallado al modelo junto con la solicitud de corrección, en un ciclo similar al descrito en (?) para la generación de consultas NRQL.

3.4.3. Comparativa y Roles en la Arquitectura Híbrida

La arquitectura híbrida propuesta integra tres tecnologías fundamentales descritas anteriormente.

La Tabla 3.2 resume las principales diferencias entre las tres tecnologías y sus roles en el sistema propuesto.

En la arquitectura del robot Justina, estas tres tecnologías se complementan:

- **CLIPS** actúa como núcleo de planificación segura, manejando tareas críticas y bien definidas donde la predictibilidad es esencial.

- **Qwen2.5-0.5B** opera como traductor local para comandos rutinarios, proporcionando respuestas rápidas y privadas sin dependencia de la nube.
- **ChatGPT** se reserva para situaciones complejas o novedosas donde la flexibilidad semántica de un modelo de gran escala es necesaria, aceptando la latencia adicional en aras de la adaptabilidad.

Esta integración sinérgica permite aprovechar las fortalezas de cada tecnología mientras se mitigan sus limitaciones individuales, constituyendo el núcleo de la propuesta híbrida de esta tesis.

Tabla 3.2: Comparativa y roles en la arquitectura híbrida

Aspecto	CLIPS	ChatGPT	Qwen2.5-0.5B
Paradigma	Simbólico (reglas)	Subsimbólico (red neuronal)	Subsimbólico (red neuronal)
Ejecución	Local (embebido)	Nube (API)	Local (edge)
Requisitos	CPU, < 10 MB RAM	GPU en servidor, internet	CPU, < 2 GB RAM
Determinismo	Completo	Probabilístico	Probabilístico
Explicabilidad	Alta (reglas trazables)	Baja (“caja negra”)	Baja (“caja negra”)
Flexibilidad	Baja (reglas fijas)	Alta (generalización)	Media (dominio restringido)
Costo operativo	Cero	Por token (API)	Cero
Rol en el sistema	Planificación segura, control crítico	Traducción de comandos complejos, planificación dual	Traducción local para tareas rutinarias

3.5. ChatGPT como Agente de Planificación Autónomo

La gran mejora de los modelos de lenguaje grande (LLMs) ha transformado no solo la interacción humano-robot, sino también el propio proceso de planificación de acciones en robótica autónoma. Más allá de su función como traductores de lenguaje natural a representaciones simbólicas, modelos como ChatGPT han demostrado capacidades

emergentes para actuar como agentes de planificación autónomos, generando secuencias completas de acciones a partir de objetivos de alto nivel expresados en lenguaje natural (6). Esta capacidad abre nuevas posibilidades, donde los LLMs no solo complementan, sino que pueden competir con los sistemas basados en reglas en la generación de planes, ofreciendo una flexibilidad contextual sin precedentes.

3.5.1. Capacidades de Planificación de LLMs vs. Sistemas Basados en Reglas

Tabla 3.3: Comparativa entre planificación con sistemas basados en reglas y LLMs

Dimensión	Sistemas Basados en Reglas (CLIPS)	Modelos de Lenguaje Grande (ChatGPT)
Fundamento	Razonamiento simbólico, lógica formal	Inferencia probabilística, patrones estadísticos
Generación de planes	Descomposición recursiva basada en métodos y operadores	Generación autorregresiva basada en contexto
Conocimiento	Explicitado en reglas (declarativo)	Implícito en los pesos de la red neuronal
Adaptabilidad	Baja (requiere modificación manual de reglas)	Alta (generaliza a situaciones novedosas)
Explicabilidad	Alta (trazabilidad de reglas activadas)	Baja (opacidad de la red neuronal)
Seguridad	Verificable formalmente	No verificable formalmente
Manejo de ambigüedad	Ninguno (la entrada debe ser estructurada)	Alta (interpretación contextual)
Actualización del conocimiento	Manual (añadir/modificar reglas)	Fine-tuning o prompting contextual
Consistencia	Determinista	Estocástica (puede variar entre ejecuciones)

Los sistemas basados en reglas (RBS) y los modelos de lenguaje grande (LLMs) representan paradigmas fundamentalmente distintos para la generación de planes en robótica. Mientras que los primeros operan mediante razonamiento simbólico explícito

sobre una base de reglas predefinidas, los segundos generan planes mediante inferencia probabilística a partir de patrones aprendidos durante su entrenamiento (2). La Tabla 3.3 resume las principales diferencias entre ambos enfoques, que se analizan en detalle a continuación.

Sistemas Basados en Reglas: Rigidez Predecible

Los sistemas basados en reglas, como CLIPS, generan planes mediante un proceso de razonamiento que opera sobre un conjunto de reglas y hechos explícitamente definidos (3). Este proceso se caracteriza por:

- **Determinismo:** Dadas las mismas condiciones iniciales, el sistema produce invariablemente el mismo plan, lo que facilita la validación y certificación del comportamiento (2).
- **Explicabilidad:** Cada decisión puede rastrearse hasta las reglas que la generaron, permitiendo auditoría y depuración sistemática (3).
- **Compleitud del conocimiento:** Requiere que todo el conocimiento necesario para la planificación esté explícitamente codificado en la base de reglas, lo que implica un esfuerzo significativo de ingeniería del conocimiento (4).
- **Falta de generalización:** Las reglas solo cubren las situaciones explícitamente contempladas; cualquier desviación puede llevar al fracaso del planificador (6).

Esta rigidez hace que los RBS sean ideales para dominios bien estructurados con comportamientos predecibles, como entornos industriales o tareas repetitivas, pero limita su aplicabilidad en entornos domésticos dinámicos donde la variabilidad es la norma.

Modelos de Lenguaje Grande: Adaptabilidad Contextual

Los LLMs como ChatGPT generan planes mediante un proceso fundamentalmente diferente: dada una descripción del objetivo y el contexto, el modelo produce autoregresivamente una secuencia de acciones, token por token, basándose en patrones aprendidos durante su entrenamiento en corpus masivos de texto (5). Este enfoque ofrece características distintivas:

- **Generalización emergente:** Los LLMs pueden generar planes para situaciones no vistas durante el entrenamiento, aprovechando su capacidad para componer patrones aprendidos de manera novedosa (6).
- **Interpretación de lenguaje natural:** Pueden procesar directamente descripciones de tareas en lenguaje natural, sin necesidad de traducción previa a repre-

sentaciones formales (42).

- **Razonamiento contextual:** Incorporan información contextual implícita (como normas sociales, preferencias comunes, o propiedades de objetos) que no está explícitamente programada (45).
- **Flexibilidad ante restricciones:** Pueden adaptar planes dinámicamente cuando se introducen nuevas restricciones o requisitos en el diálogo.

Sin embargo, esta flexibilidad conlleva riesgos significativos. La naturaleza probabilística de los LLMs implica que el mismo objetivo puede generar planes diferentes en distintas ejecuciones, y no existe garantía formal de que las acciones generadas sean seguras o factibles en el mundo físico (6). Además, la opacidad del razonamiento subyacente dificulta la auditoría y la certificación del comportamiento.

Complementariedad Fundamental

Lejos de ser enfoques opuestos, los RBS y los LLMs presentan una complementariedad fundamental que motiva su integración en arquitecturas híbridas (9):

- Los **RBS** proporcionan seguridad garantizada, explicabilidad y comportamiento determinista para tareas críticas y bien definidas.
- Los **LLMs** aportan flexibilidad semántica, adaptabilidad a situaciones novedosas y capacidad de interacción natural para tareas complejas y mal definidas.

Esta complementariedad constituye la base de la arquitectura de planificación dual propuesta en esta tesis, donde ambos motores operan de manera concurrente y un módulo supervisor selecciona o combina sus salidas según el contexto.

3.5.2. Arquitectura de Planificación Dual: CLIPS y ChatGPT como Motores Complementarios

La arquitectura de planificación dual propuesta en esta investigación integra CLIPS y ChatGPT como motores complementarios que operan en paralelo, generando planes concurrentemente y permitiendo la selección dinámica de la mejor estrategia según el contexto (9). La Figura 3.3 ilustra la estructura general de esta arquitectura, cuyos componentes principales se describen a continuación.

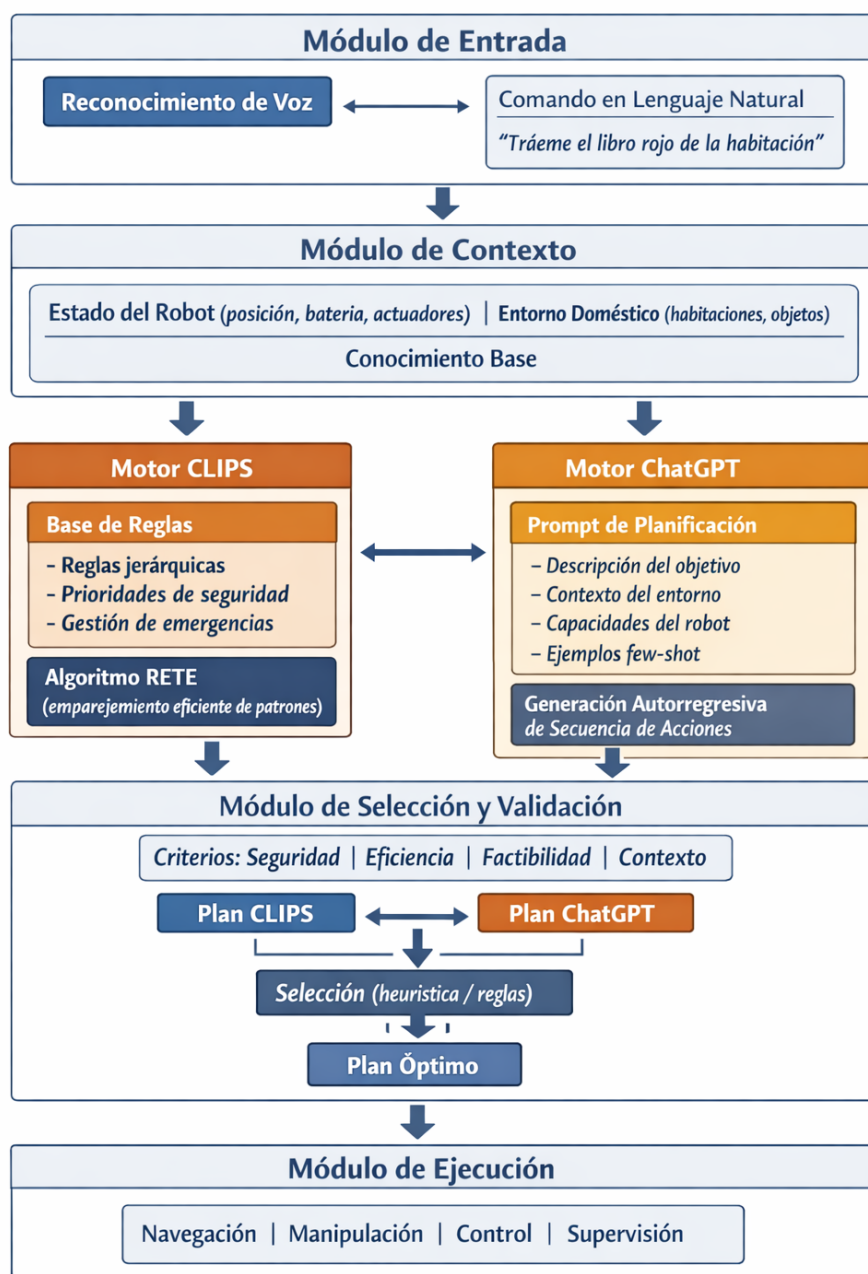


Figura 3.3: Arquitectura de planificación dual CLIPS-ChatGPT.

Principios de la Planificación Dual

La arquitectura de planificación dual se fundamenta en los siguientes principios (6):

1. **Procesamiento Concurrente:** Ambos motores operan en paralelo, generando planes simultáneamente a partir del mismo objetivo y contexto. Esto permite que la latencia de generación no sea aditiva, sino que el sistema pueda seleccionar el primer plan válido o esperar la finalización de ambos según la estrategia de selección.
2. **Independencia de Representación:** CLIPS opera con representaciones simbólicas (hechos y reglas), mientras que ChatGPT procesa representaciones textuales. La arquitectura mantiene esta independencia mediante interfaces de traducción que convierten el contexto común al formato requerido por cada motor.
3. **Estrategias de Selección Configurables:** El módulo de selección puede implementar diferentes estrategias según los requisitos de la aplicación, como priorizar el plan de CLIPS por defecto (seguridad primero), priorizar el plan de ChatGPT para tareas novedosas (flexibilidad primero), o combinar elementos de ambos planes.

Generación de Planes con ChatGPT

Para que ChatGPT genere planes robóticos ejecutables, se requiere un diseño cuidadoso de los prompts que incluyan toda la información contextual necesaria (6). La estructura típica del prompt de planificación incluye:

1. **Especificación del rol:** Se establece explícitamente que el modelo actúa como planificador robótico: "Eres un planificador de acciones para un robot de servicio doméstico."
2. **Descripción del entorno:** Se proporciona información sobre las habitaciones, objetos disponibles, y relaciones espaciales, en formato de texto estructurado.
3. **Capacidades del robot:** Se enumeran las acciones primitivas que el robot puede ejecutar (navegar, buscar, agarrar, soltar, etc.), junto con sus precondiciones.
4. **Objetivo en lenguaje natural:** Se presenta la tarea que debe cumplirse, expresada en lenguaje natural.
5. **Ejemplos few-shot:** Se incluyen ejemplos de objetivos similares y los planes generados, para guiar al modelo hacia el formato deseado (?).
6. **Restricciones de formato:** Se especifica que el plan debe producirse como una secuencia numerada de acciones, cada una con parámetros explícitos.

Un ejemplo de prompt estructurado para planificación con ChatGPT es el siguiente:

Eres un planificador de acciones para un robot de servicio doméstico.

Entorno:

- Habitaciones: cocina, sala, comedor, habitacion_principal
- Objetos: vaso_rojo (en cocina, mesa); libro (en sala, estante)
- Ubicación actual del robot: entrada

Capacidades del robot:

- goto(habitacion)
- find_object(objeto)
- grasp(objeto)
- release(objeto)
- bring_object(objeto, destino)

Objetivo: "Tráeme el vaso rojo que está en la cocina a la sala"

Ejemplo de plan válido:

1. goto(cocina)
2. find_object(vaso_rojo)
3. grasp(vaso_rojo)
4. goto(sala)
5. release(vaso_rojo)

Genera un plan para el objetivo indicado.

Responde únicamente con la secuencia numerada de acciones.

Este enfoque de prompting estructurado ha demostrado ser efectivo para generar planes coherentes y ejecutables en entornos robóticos (6).

3.5.3. Mecanismos de Selección y Validación de Planes

La planificación dual introduce un desafío crítico: cómo seleccionar el plan más adecuado cuando los dos motores generan propuestas potencialmente divergentes. Para abordar este desafío, la arquitectura implementa un módulo de selección y validación que evalúa los planes generados según múltiples criterios y aplica heurísticas de decisión.

Criterios de Evaluación

Los planes generados por CLIPS y ChatGPT se evalúan según los siguientes criterios (2,6):

Tabla 3.4: Criterios de evaluación para selección de planes

Criterio	Descripción	Métrica
Seguridad	El plan no debe incluir acciones que puedan causar daño al robot, al entorno o a las personas.	Evaluación de reglas de seguridad (verificación de precondiciones críticas)
Factibilidad	Todas las acciones deben ser ejecutables por el robot dadas sus capacidades físicas.	Verificación de acciones contra capacidades declaradas
Consistencia	La secuencia de acciones debe respetar los efectos de acciones previas.	Verificación de encañamiento de precondiciones
Eficiencia	El plan debe minimizar el número de acciones o el tiempo estimado de ejecución.	Longitud del plan, estimación de tiempo por acción
Adecuación contextual	El plan debe respetar normas sociales implícitas (ej., no interrumpir actividades humanas).	Evaluación mediante reglas sociales o LLM supervisor

Estrategias de Selección

El módulo de selección puede implementar diversas estrategias según los requisitos de la aplicación:

1. **Selección por defecto priorizando CLIPS:** El plan de CLIPS se ejecuta por defecto. El plan de ChatGPT solo se utiliza cuando CLIPS no puede generar un plan (base de reglas incompleta) o cuando el plan de CLIPS viola criterios de eficiencia (longitud excesiva). Esta estrategia prioriza la seguridad y previsibilidad.
2. **Selección por defecto priorizando ChatGPT:** El plan de ChatGPT se ejecuta por defecto para maximizar adaptabilidad. El plan de CLIPS se utiliza como respaldo cuando el plan de ChatGPT no supera la validación de seguridad. Esta estrategia es adecuada para entornos muy dinámicos donde la rigidez de CLIPS sería limitante.

3. **Selección basada en voto ponderado:** Cada plan recibe una puntuación agregada según los criterios de evaluación. Se selecciona el plan con mayor puntuación. Los pesos de los criterios pueden configurarse según la aplicación.
4. **Fusión de planes:** En lugar de seleccionar un plan completo, el módulo puede fusionar elementos de ambos planes, por ejemplo, utilizando la secuencia de navegación de CLIPS (más segura) y la estrategia de manipulación de ChatGPT (más adaptativa).

Validación de Seguridad Pre-Ejecución

Independientemente de la estrategia de selección, todos los planes pasan por una fase de validación de seguridad pre-ejecución antes de ser enviados al módulo de ejecución (45). Esta validación incluye:

- **Verificación de precondiciones:** Para cada acción en la secuencia, se verifica que sus precondiciones se cumplan en el estado esperado.
- **Detección de acciones inseguras:** Se aplican reglas de seguridad explícitas (ej., no acercarse a áreas restringidas, no manipular objetos frágiles sin verificación) que pueden rechazar acciones específicas.
- **Simulación de ejecución:** Se ejecuta una simulación rápida del plan en un modelo simplificado del entorno para detectar conflictos espaciales o temporales.
- **Validación de límites físicos:** Se verifican restricciones como alcance del brazo, capacidad de carga, o niveles de batería.

En caso de que un plan no supere la validación de seguridad, se recurre al plan alternativo (si está disponible) o se solicita una replanificación con restricciones adicionales.

3.5.4. Ventajas y Riesgos de la Planificación con LLMs en Robótica

La integración de LLMs como motores de planificación autónomos en sistemas robóticos ofrece ventajas significativas, pero también introduce riesgos que deben gestionarse cuidadosamente.

Ventajas

1. **Flexibilidad semántica sin precedentes:** Los LLMs pueden interpretar objetivos expresados en lenguaje natural con alta variabilidad léxica y sintáctica,

adaptando sus respuestas al contexto (6). Un usuario puede decir "trae lo que está en la mesa de la cocina" y el modelo inferirá que se refiere al objeto que estaba allí previamente.

2. **Razonamiento de sentido común:** Incorporan conocimiento del mundo que no está explícitamente programado, como saber que un vaso suele estar en la cocina o que los libros no deben dejarse en el suelo (42).
3. **Adaptación dinámica a restricciones:** Pueden replanificar en tiempo real cuando se introducen nuevas restricciones (ej., "pero no pases por la sala porque hay personas comiendo") sin necesidad de reprogramación (45).
4. **Reducción de la ingeniería del conocimiento:** Eliminan la necesidad de codificar explícitamente todas las posibles situaciones y respuestas, reduciendo el esfuerzo de desarrollo y mantenimiento.
5. **Interacción explicativa:** Pueden generar explicaciones de sus decisiones en lenguaje natural, mejorando la transparencia para usuarios no técnicos (6).

Riesgos y Desafíos

1. **Falta de garantías de seguridad:** No existe un marco formal para verificar que los planes generados por LLMs sean seguros en todas las circunstancias. Un modelo puede generar acciones aparentemente razonables que, en el contexto físico, resulten peligrosas (6).
2. **Comportamiento no determinista:** La misma entrada puede generar planes diferentes en distintas ejecuciones, dificultando la predicción del comportamiento y la certificación del sistema.
3. **Alucinaciones (hallucinations):** Los LLMs pueden generar acciones o referencias a objetos inexistentes, con total confianza y plausibilidad superficial (6). Por ejemplo, un modelo podría sugerir buscar un objeto en una habitación que no existe en el entorno.
4. **Dependencia de la calidad del prompt:** La calidad del plan generado es altamente sensible a la formulación del prompt. Pequeñas variaciones pueden llevar a planes radicalmente diferentes.
5. **Latencia y recursos:** La inferencia con LLMs de gran escala requiere recursos computacionales significativos y puede introducir latencias inaceptables para aplicaciones de tiempo real.
6. **Privacidad de datos:** El uso de APIs externas implica enviar información con-

textual potencialmente sensible a servidores de terceros.

7. **Sesgos y equidad:** Los LLMs pueden heredar sesgos presentes en los datos de entrenamiento, que podrían manifestarse como comportamientos inapropiados en contextos sociales (6).

Estrategias de Mitigación

Para gestionar estos riesgos, la arquitectura propuesta implementa múltiples capas de mitigación:

- **Validación por reglas de seguridad:** Todos los planes generados por ChatGPT son filtrados por un conjunto de reglas de seguridad en CLIPS que bloquean acciones peligrosas.
- **Ejecución supervisada:** Un módulo supervisor monitorea la ejecución y puede interrumpir acciones que se desvíen del comportamiento esperado.
- **Simulación previa:** Los planes se simulan en un entorno virtual antes de la ejecución física para detectar problemas potenciales.
- **Uso limitado a tareas no críticas:** En la implementación inicial, ChatGPT se utiliza principalmente para tareas donde el fallo no implica riesgos de seguridad significativos.
- **Fallback a CLIPS:** En caso de que ChatGPT no genere un plan válido o este no supere la validación, el sistema recurre automáticamente al plan generado por CLIPS.

Esta combinación de flexibilidad y control permite aprovechar las ventajas de los LLMs mientras se mantienen los estándares de seguridad requeridos en robótica de servicio.

Capítulo 4

Metodología: Diseño del Sistema Híbrido CLIPS-ChatGPT/Qwen

4.1. Diseño del Sistema Basado en Reglas con CLIPS

4.1.1. Definición del Conjunto de Reglas Jerárquicas

Especificación de las reglas organizadas por niveles de prioridad, donde las reglas de seguridad (evitación de colisiones, gestión de emergencias) tienen máxima prioridad sobre las operativas.

4.1.2. Priorización de Acciones Críticas y Gestión de Emergencias

Diseño de reglas específicas para escenarios de fallo y situaciones críticas, asegurando respuestas predecibles y seguras.

4.1.3. Representación del Conocimiento y Hechos en la Base de Datos de CLIPS

Estructuración del conocimiento del entorno doméstico en hechos CLIPS iniciales que sirven como base para el razonamiento.

Descripción de la base de conocimiento inicial incluye:

- *Topología del entorno*: habitaciones, conexiones, zonas navegables.
- *Taxonomía de objetos*: categorías, subcategorías, propiedades heredables.
- *Relaciones espaciales*: contención, adyacencia, orientación.
- *Capacidades del robot*: acciones posibles, restricciones físicas.

4.2. Integración de ChatGPT/Qwen como Sistema Complementario

4.2.1. Funciones Asignadas al Módulo de Lenguaje Natural

4.2.2. Protocolo de Comunicación entre CLIPS y los Modelos de Lenguaje

4.2.3. Mecanismos de Seguridad y Validación para las Respuestas del LLM

Implementación de verificaciones para asegurar que los planes generados por LLMs cumplen con constraints de seguridad y factibilidad robótica.

4.3. Implementación de Planificación Dual

4.3.1. Diseño del Módulo de Planificación con ChatGPT

4.3.2. Protocolos de Prompting para Generación de Planes Robóticos

4.3.3. Mecanismo de Selección entre Planes CLIPS vs. ChatGPT

4.3.4. Validación y Simulación de Planes Antes de Ejecución

Integración con el Robot Justina

- *Procesamiento de Entrada de Voz*.
- *Ejecución de Planes en el Entorno Físico*.

- Mecanismos para transformar los planes publicados en los tópicos de resultados en comandos ejecutables por los actuadores del robot (navegación, manipulación de objetos, etc.)
- *Retroalimentación y Aprendizaje Incremental.*

Capítulo 5

Implementación

- 5.1. Entornos de Desarrollo: Simulación y Plataforma Física
- 5.2. Implementación del Motor de Reglas en CLIPS
- 5.3. Desarrollo del Módulo de Integración
- 5.4. Configuración de la Interfaz con API de OpenAI para ChatGPT o Modelo Qwen Local
- 5.5. Casos de Prueba para Validar la Interacción entre los Módulos

Capítulo 6

Escenarios de Validación y Experimentación

6.1. Diseño de Experimentos en Entornos Controlados

6.1.1. Escenario 1: Ejecución de Tareas Predefinidas (solo CLIPS)

6.1.2. Escenario 2: Gestión de Órdenes Imprecisas o Novedosas (CLIPS + ChatGPT/Qwen)

6.1.3. Escenario 3: Respuesta a Eventos Inesperados o Fallos

6.1.4. Escenario 4: Planificación Dual para Tareas Complejas (CLIPS vs. ChatGPT)

- Comparativa de eficiencia en generación de planes
- Evaluación de robustez ante escenarios novedosos
- Análisis de seguridad en planes generados por LLMs

6.2. Métricas de Evaluación

6.2.1. Métricas Cuantitativas: Tiempo de Ejecución, Tasa de Éxito, Uso de Recursos Computacionales

6.2.2. Métricas Cualitativas: Robustez, Interpretabilidad y Fluidez en la Interacción Humano-Robot

Capítulo 7

Análisis de Resultados

- 7.1. Comparativa del Rendimiento del Sistema Solo-CLIPS vs. el Sistema Híbrido
- 7.2. Evaluación de la Efectividad de ChatGPT/Qwen en las Diferentes Funciones Asignadas
- 7.3. Discusión de Limitaciones y Errores
- 7.4. Análisis de la Escalabilidad y el Coste Computacional de la Integración

Capítulo 8

Discusión

- 8.1. Interpretación de los Resultados en el Contexto de los Objetivos
- 8.2. Ventajas y Desventajas del Enfoque Híbrido Propuesto
- 8.3. Implicaciones para la Seguridad y Certificación en Entornos Regulados
- 8.4. Límites Éticos y Prácticos del Uso de ChatGPT/Qwen en Robótica

Capítulo 9

Contribuciones y Relevancia

- 9.1. Contribución Técnica: Marco Híbrido Escalable y Documentado
- 9.2. Relevancia Práctica: Aplicaciones en Salud, Industria y Logística
- 9.3. Relevancia Académica: Benchmark para Sistemas Rule-Based vs. Híbridos

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Conclusiones Principales

10.2. Propuestas de Trabajo Futuro

- Fine-tuning de un LLM de código abierto para dominio específico
- Mejora de los mecanismos de seguridad y verificación
- Exploración de arquitecturas de integración más profundas

Referencias

- [1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.
- [3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.
- [4] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, “Chatgpt for robotics: Design principles and model abilities,” *Microsoft Research*, 2023.
- [7] NASA Johnson Space Center, “Clips reference manual,” tech. rep., NASA, 1991.
- [8] B. Pell *et al.*, “An autonomous spacecraft agent prototype,” in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.
- [9] J. Savage and otros, “Virbot: A modular architecture for service robots with hybrid planning,” in *RoboCup Symposium*, 2024.
- [10] L. Cheng, Y. Mao, and T. Ward, “Editorial: Neural network models in autonomous robotics,” *Frontiers in Neurorobotics*, 2025.
- [11] R. Raj and A. Kos, “An extensive study of convolutional neural networks: Applications in computer vision for improved robotics perceptions,” *Sensors*, 2025.
- [12] A. Sadek, *Building autonomous agents with hybrid navigation policies*. PhD thesis, Université de Technologie de Compiègne, 2024.

- [13] Alibaba Cloud Qwen Team, “Qwen official github repository,” 2026.
- [14] OpenAI, “Openai developer documentation and model guides,” 2026.
- [15] N. Houlsby, A. Giurciu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.
- [16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [17] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.
- [18] A. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.
- [19] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, “Neural-symbolic learning and reasoning: A survey and interpretation,” *arXiv preprint arXiv:1711.03902*, 2017.
- [20] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [21] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [22] D. Nau, T.-C. Au, O. Ilghami, U. Kuter, J. W. Murdock, D. Wu, and F. Yaman, “Shop2: An htn planning system,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 379–404, 2003.

- [23] K. Erol, J. Hendler, and D. S. Nau, “Htn planning: Complexity and expressivity,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 1123–1128, 1994.
- [24] I. Georgievski and M. Aiello, “An overview of hierarchical task network planning,” *Journal of Autonomous Agents and Multi-Agent Systems*, vol. 30, no. 4, pp. 709–752, 2016.
- [25] R. E. Fikes and N. J. Nilsson, “Strips: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [26] H. Geffner, “Model-free, model-based, and general intelligence,” *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 10–17, 2020.
- [27] R. Lallement, L. de Silva, and R. Alami, “Hatp: An htn planner for robotics,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 10–18, 2014.
- [28] L. de Silva, R. Lallement, and R. Alami, “The hatp hierarchical planner: Formalisation and an initial study of its usability and practicality,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5845–5852, 2016.
- [29] L. de Silva and R. Alami, “Dealing with on-line human-robot negotiations in hierarchical agent-based task planner,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 67–75, 2017.
- [30] R. Alami, R. Chatila, A. Clodic, *et al.*, “On human-aware task and motion planning,” *Robotics and Autonomous Systems*, vol. 61, no. 8, pp. 804–813, 2013.
- [31] X. Luo, S. Xu, R. Liu, and C. Liu, “Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6142–6149, 2024.
- [32] E. Gat, “On three-layer architectures,” in *Artificial Intelligence and Mobile Robots*, pp. 195–210, MIT Press, 1998.
- [33] F. Morales *et al.*, “Automated planning and scheduling in robot-aided rehabilitation: A review,” *Journal of NeuroEngineering and Rehabilitation*, vol. 22, no. 1, 2025.

- p. 45, 2025.
- [34] W. Pang, H. Li, Q. Huang, P. Li, and X. Ma, “Ontology-based mission planning for unmanned systems: A review,” *Journal of Unmanned Systems*, vol. 10, no. 2, pp. 112–128, 2022.
 - [35] Middlesex University, “Recognise act cycle,” 2023.
 - [36] B. L. Donnell, “Object/rule integration in clips,” *Expert Systems*, vol. 11, no. 1, pp. 29–45, 1994.
 - [37] F. J. Fanning, “An evaluation of an ada implementation of the rete algorithm for embedded flight processors,” *Air Force Institute of Technology Theses and Dissertations*, 1990.
 - [38] University of Žilina, “Production rules,” 2023.
 - [39] ScienceDirect, “Production systems and rules,” 2023.
 - [40] C. L. Forgy, “Rete: A fast algorithm for the many pattern/many object pattern match problem,” *Artificial Intelligence*, vol. 19, no. 1, pp. 17–37, 1982.
 - [41] C. L. Forgy, *On the efficient implementation of production systems*. PhD thesis, Carnegie-Mellon University, 1979.
 - [42] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee, “Robotic pick-and-place with natural language commands,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 4615–4622, 2020.
 - [43] L. Albert, “Average case complexity analysis of rete pattern-match algorithm, and average size of join in databases,” in *Foundations of Software Technology and Theoretical Computer Science*, vol. 405 of *Lecture Notes in Computer Science*, pp. 223–241, Springer, 1989.
 - [44] GitHub, “Attention is all you need - paper reading notes,” 2023.
 - [45] C. de desarrolladores de Alibaba Cloud, “Transformer modelos de lenguaje de nuestro tiempo: El desarrollo y la evolución tecnológica de los modelos de lenguaje a gran escala,” 2026.
 - [46] systems analysis.ru, “Transformer-architektur - der transformer,” 2025.
 - [47] A. Kashyap, “Recurrent memory-augmented transformers with chunked attention for long-context language modeling,” *arXiv preprint arXiv:2507.00453*, 2025.

- [48] D. Pollini, B. V. Guterres, R. S. Guerra, and R. B. Grando, “Reducing latency in llm-based natural language commands processing for robot navigation,” 2025.
- [49] C. Blog, “Qwen2.5-0.5b caso práctico: Guía para la construcción de un robot guía inteligente.,” 2026.